REALISMO

"El verdadero realismo consiste en revelar las cosas sorprendentes que se mantienen cubiertas por el hábito y nos impiden ver"

Jean Cocteau

REALISMO

1. Ray tracing
   1.1. Modelo de iluminación transicional
   1.2. Algoritmo clásico
   1.3. Ray tracing basado en Montecarlo
   1.4. Software para ray tracing
   1.5. Efectos

2. Radiosidad

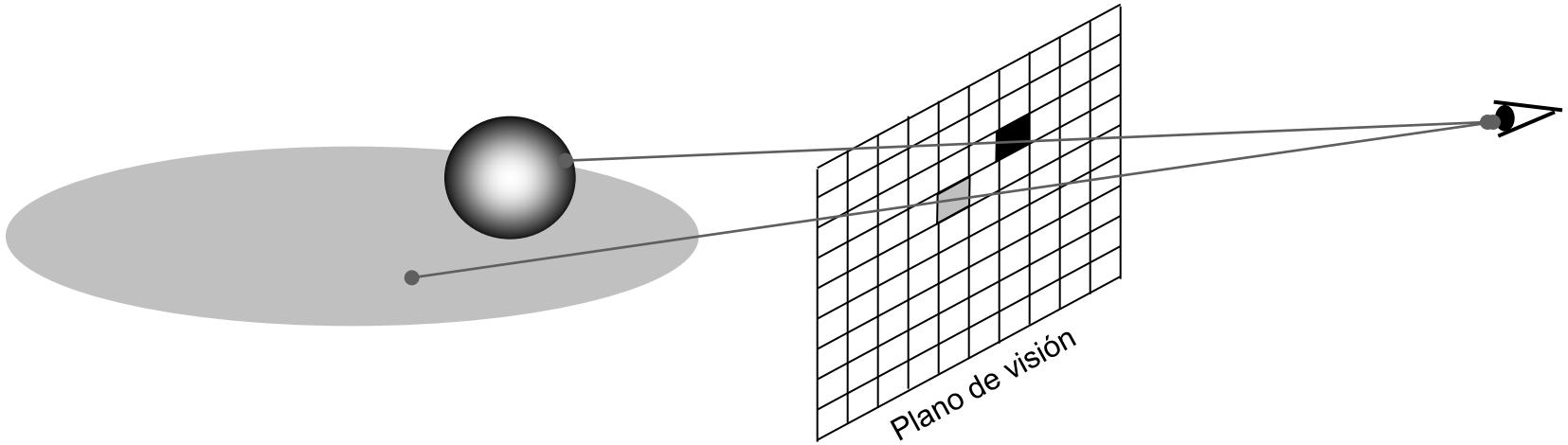
3. Photon mapping

1. Ray tracing

- Gauss: tres ideas sobre la luz:
  - La Luz viaja en línea recta
  - Los rayos de luz no interfieren entre sí cuando se cruzan
  - Los rayos de luz viajan desde las fuentes de luz al ojo (pero la física es invariante en una situación inversa)
- El ray tracing es una extensión del algoritmo de ray casting de Appel (1968) y se basa en la óptica geométrica
- Ray tracing recursivo (reflexiones y refraccciones) de Whitted 1979
- Efectos físicos que se simula :
  - Superficies difusas y especulares, sombras arrojadas, transmisión y reflexión

1. Ray tracing

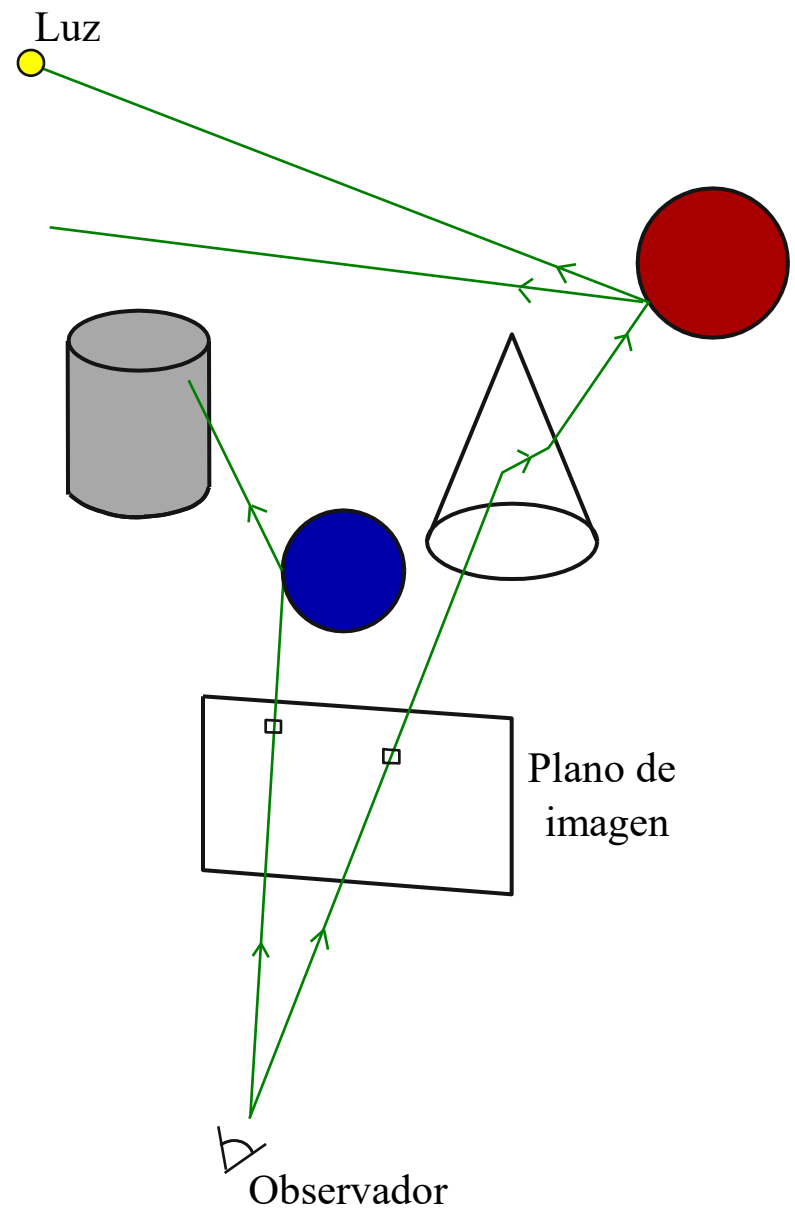
- Algoritmo de eliminación de partes ocultas de Ray Casting.
  - El método funciona en el espacio imagen
  - Para cada pixel de la imagen se crea un rayo (un par formado por un punto y un vector) con origen en el observador y que pase por el centro de dicho pixel.
  - Se encuentran las intersecciones con los objetos de la escena y nos quedamos con la más cercana al observador. Esta intersección determina el color del píxel.



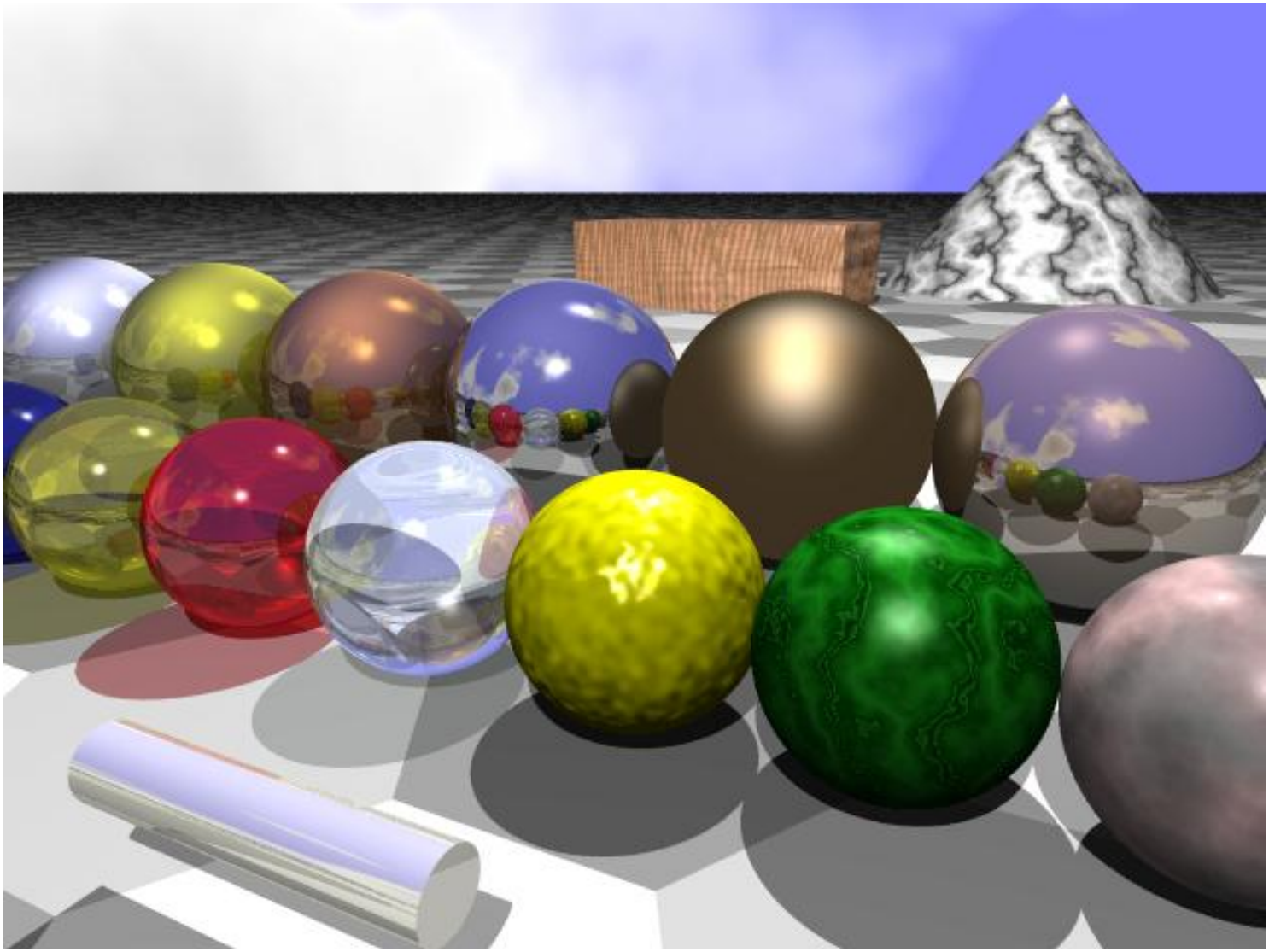
1. Ray tracing

- Permite de una sola vez generar imágenes con eliminación de partes ocultas, proyección en perspectiva, sombras y simular modelos de iluminación empíricos globales.

- Consiste en simular el recorrido de un rayo de luz en sentido inverso, desde el ojo de un observador hacia una escena (Algoritmo de Whitted)



1. Ray tracing



1. Ray tracing

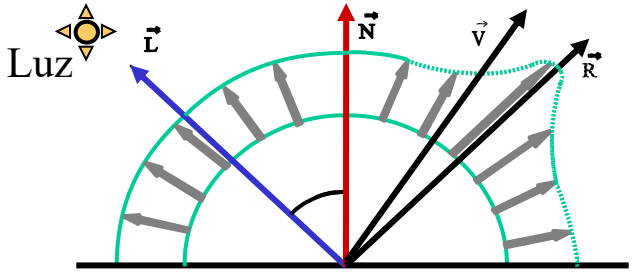
1.1. Modelo de iluminación transicional

- Están basados en el transporte de la luz. Son modelos de iluminación globales al considerar que las superficies de los objetos pueden estar iluminados por la luz procedente de otros por reflexión especular y refracción (transmisión).

- De forma general (Whitted):

$$I(\lambda) = \text{ambiente} + S(\text{difusa} + \text{especular}) + \text{reflexión} + \text{transmisión}$$

$$I_\lambda = I_{a\lambda} K_{a\lambda} + \sum_i S_i I_{L(i)\lambda} \left(K_{d\lambda} (\vec{N}\vec{L}_i)_+ + K_s (\vec{R}_i \vec{V})_+^n \right) + I_{r\lambda} K_s + I_{t\lambda} K_t$$

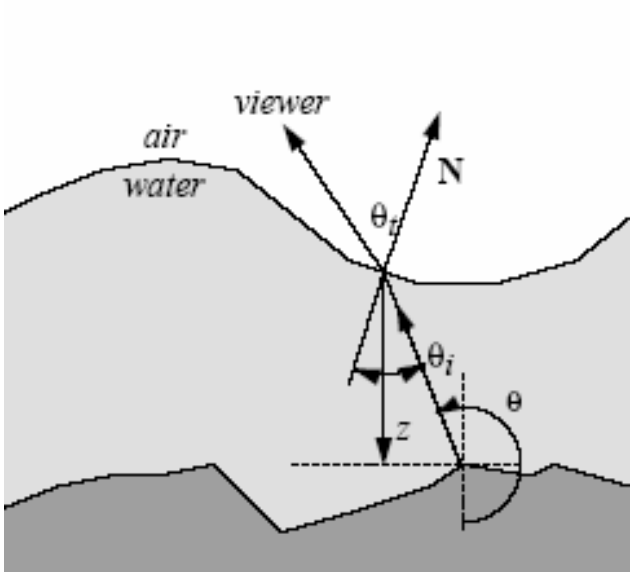


1. Ray tracing

1.1. Modelo de iluminación transicional (reflexiones)

1. Ray tracing

1.1. Modelo de iluminación transicional (refracciones)



1. Ray tracing

1.1. Modelo de iluminación transicional (sombras)

- Cada fuente de luz se considera como punto de observación

$$I_\lambda = I_{a\,\lambda} K_{a\lambda} + \sum_i S_i I_{L(i)\lambda} \left(K_{d\lambda} \left(\vec{N}\,\vec{L}_i\right)_+ + K_s \left(\vec{R}_i\,\vec{V}\right)_+^n \right) + \ldots \quad \text{con} \quad S_i \in [0,1]$$

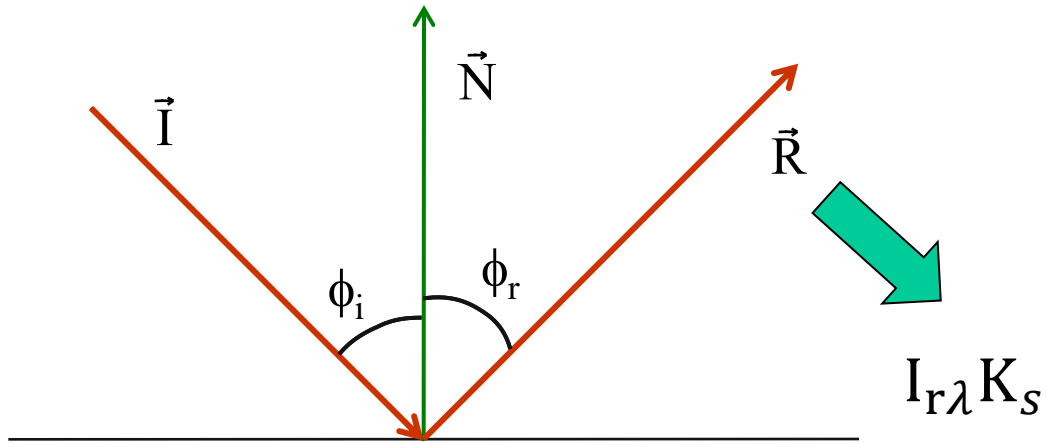
1. Ray tracing

1.1. Modelo de iluminación transicional (reflexiones)

- Se añade un término de intensidad reflejada que se calcula lanzando un rayo reflejado con respecto al incidente (el punto de intersección se considera como punto de observación)

$$I_\lambda = I_{a\lambda} K_{a\lambda} + \sum_i S_i I_{L(i)\lambda} \left(K_{d\lambda} (\vec{N} \vec{L}_i)_+ + K_s \left(\vec{R}_{L(i)} \vec{V} \right)_+^n \right) + I_{r\lambda} K_s$$

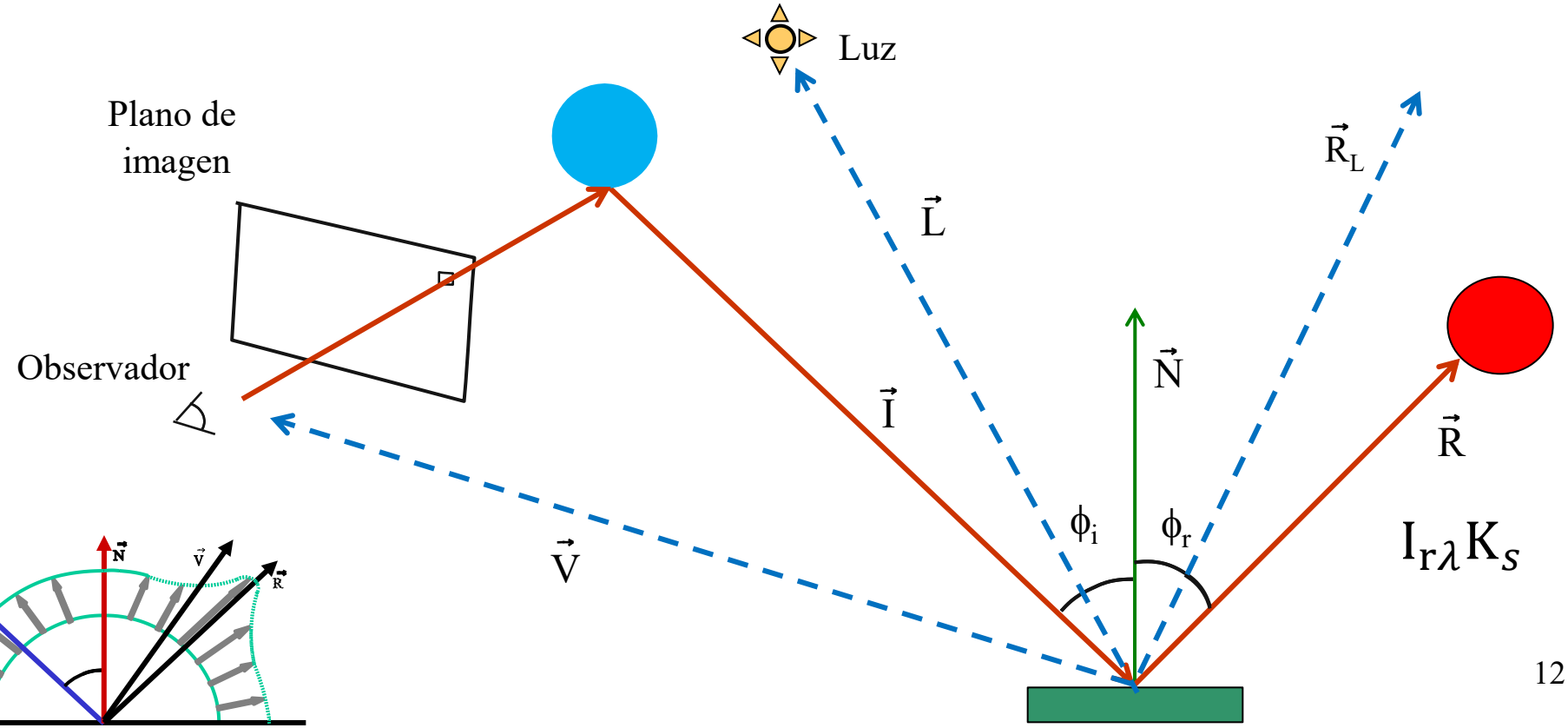
- Dirección del rayo reflejado: R = I - 2 (N.I) N

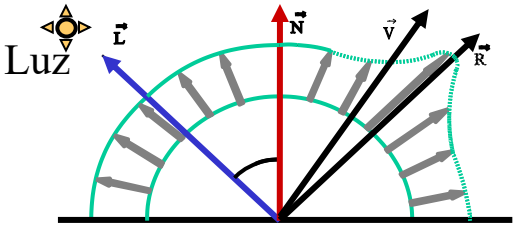


1. Ray tracing

1.1. Modelo de iluminación transicional (reflexiones)

$$I_\lambda = I_{a\lambda} K_{a\lambda} + \sum_i S_i I_{L(i)\lambda} \left(K_{d\lambda} \left(\vec{N}\,\vec{L}_i\right)_+ + K_s \left(\vec{R}_{L(i)}\,\vec{V}\right)_+^n \right) + I_{r\lambda} K_s$$





1. Ray tracing

1.1. Modelo de iluminación transicional (refracciones)

- Si la luz alcanza un material traslucido o transparente se propaga por su interior
- La dirección del rayo trasmitido se puede calcular de dos formas:
  - Se ignora la refracción: los rayos no cambian de dirección:

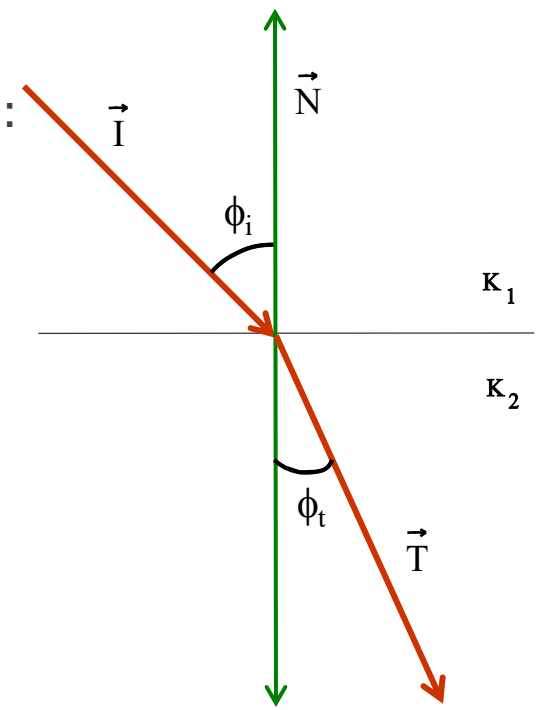
$$\vec{T} = \vec{I}$$

  - Se usa la ley de Snell o Snelius:

$$\vec{T} = \frac{K_1}{K_2}\left(\left\{-\sqrt{\left(\vec{N}\cdot\vec{I}\right)^2 + \left(\frac{K_1}{K_2}\right)^2 - 1} - \vec{N}\cdot\vec{I}\right\}\vec{N} + \vec{I}\right)$$

Con solución cuando:

$$\left(\vec{N}\cdot\vec{I}\right)^2 > 1 - \left(\frac{K_1}{K_2}\right)^2$$



- Hay reflexión total con $K_2<K_1$. El ángulo es ϕ_{rt}=arcoseno(K_2 / K_1)

1. Ray tracing

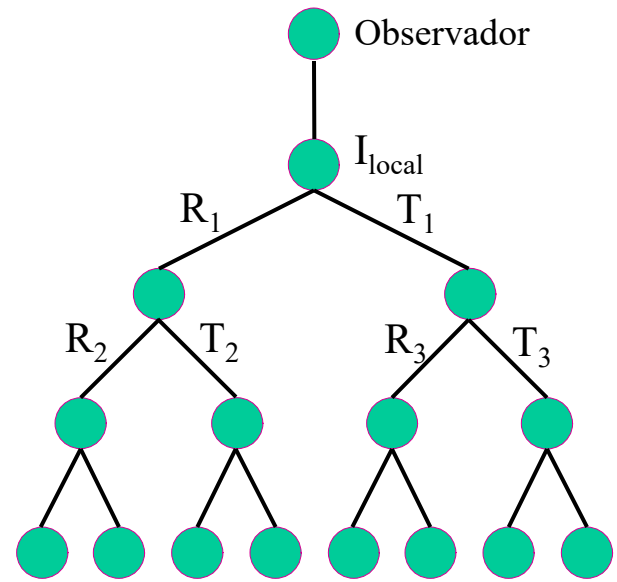
1.2. Algoritmo

- El uso de la recursión es la forma obvia de implementar un ray tracer
- En cada punto de intersección de un rayo con un objeto se consideran tres nuevos tipos de rayo: rayos hacia las fuentes de luz, un rayo reflejado y un rayo transmitido.

$$I = I_{local} + K_s\, I_{reflejada} + K_t\, I_{trasmitida}$$

$$I_{reflejada} = I'_{local} + K'_s\, I'_{reflejada} + K'_t\, I'_{trasmitida}$$

$$I_{trasmitida} = I''_{local} + K''_s\, I''_{reflejada} + K''_t\, I''_{trasmitida}$$

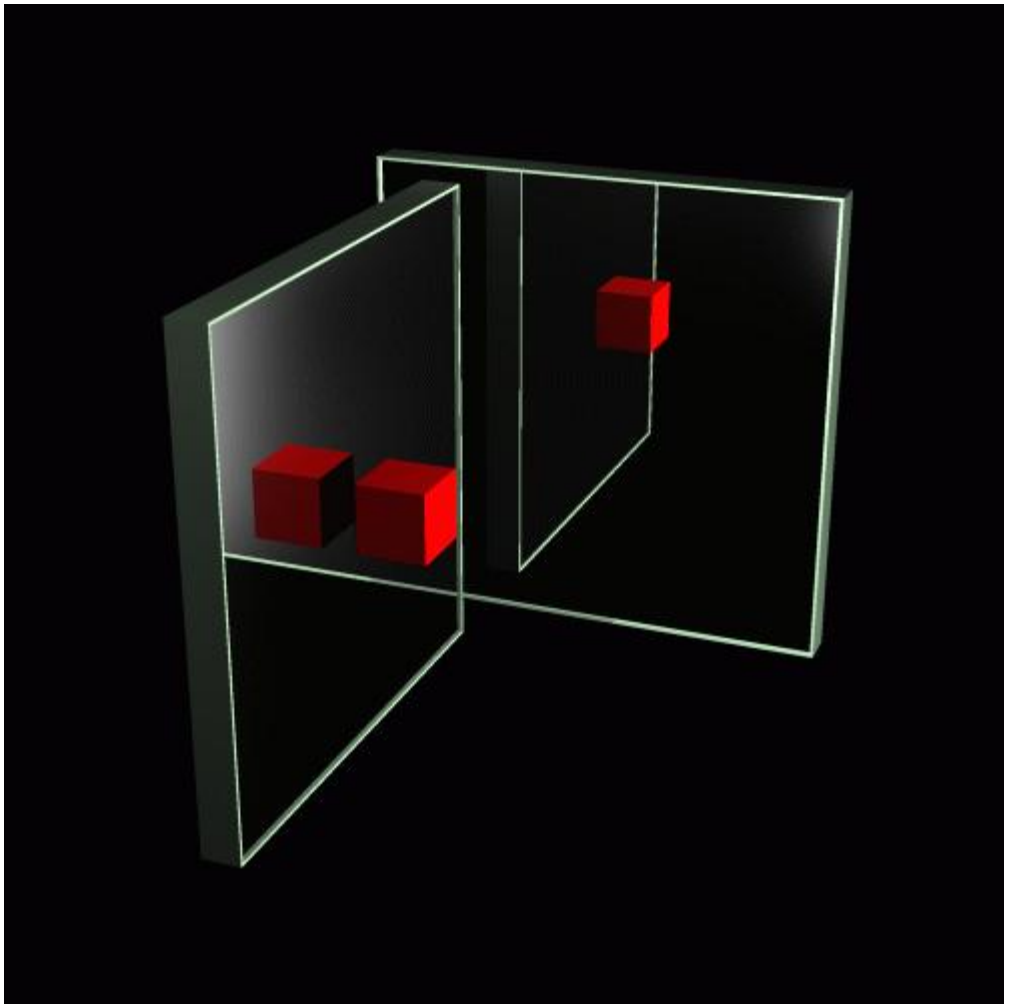


- Este proceso recursivo tiene cuatro mecanismos de parada

1. Ray tracing

1.2. Algoritmo

- Ejemplo de necesidad de mecanismos de parada



1. Ray tracing

1.2. Algoritmo. Función principal

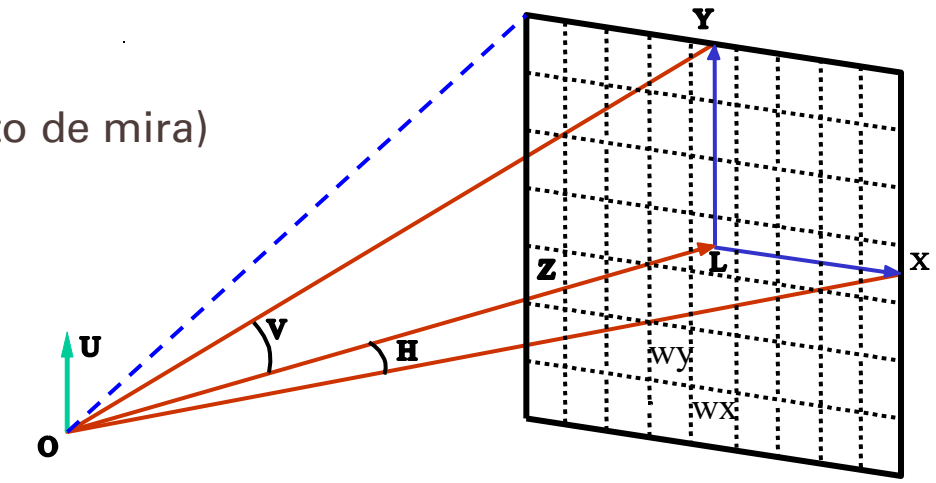
```
Ray_Tracer (Punto3D inicio, Punto3D direccion,
            Profundidad p, Color *c)
{
Punto3D  punto_inter, direccion_r, direccion_t;
Color    color_l,  color_r,  color_t,  color_aux;
Objeto   objeto;
int      i;

if (p >MAXPROFUNDIDAD) *c=NEGRO;
else
 {If (Calcular_Interseccion(inicio, direccion,
                            &objeto, &punto_inter))
   {color_l=NEGRO;
    for ( i=0; i<N_Luces; i++)
        {Iluminacion_Local(objeto, punto_inter, color_aux);
         color_l=color_l+color_aux;
        }
    if (Traslucido(objeto) && !Mate(objeto)

 {Direccion_Reflexion(objeto,punto_inter,&direccion_r);

Direccion_Trasmision(objeto,punto_inter,&direccion_t);
         Ray_Tracer (punto_inter, direccion_r, p+1, &color_r);
         Ray_Tracer (punto_inter, direccion_t, p+1, &color_t);
        }
```

```
    if (Traslucido(objeto) && Mate(objeto))
        {Direccion_Trasmision(objeto, punto_inter,&direccion_
         Ray_Tracer (punto_inter, direccion_t, p+1, &color_t);
         color_r=NEGRO;
        }
    if (!Traslucido (objeto) && !Mate(objeto))
            {Direccion_Reflexion (objeto, punto_int
&direccion_r);
         Ray_Tracer (punto_inter, direccion_r, p+1, &color_r);
         color_t=NEGRO;
        }
    c->r=color_l.r+objeto.kr_r*color_r.r+objeto.kt_r*color_t.r;
    c->g=color_l.g+objeto.kr_g*color_r.g+objeto.kt_g*color_t.g
    c->b=color_l.b+objeto.kr_b*color_r.b+objeto.kt_b*color_t..
   }
   else *c=COLOR_FONDO;
 }
}
```

1. Ray tracing

1.2. Algoritmo.

- Función para calcular las direcciones de los rayos que se lanzan a la escena
  - Cámara (pinhole)
    - = O Posición del observador
    - = Z Vector al centro de la imagen (punto de mira)
    - = U vector de inclinación
    - = L centro de la imagen
    - = Y, X alto y ancho de la imagen
    - = H, V ángulos focales
    - = wx, wy tamaño de las celdillas

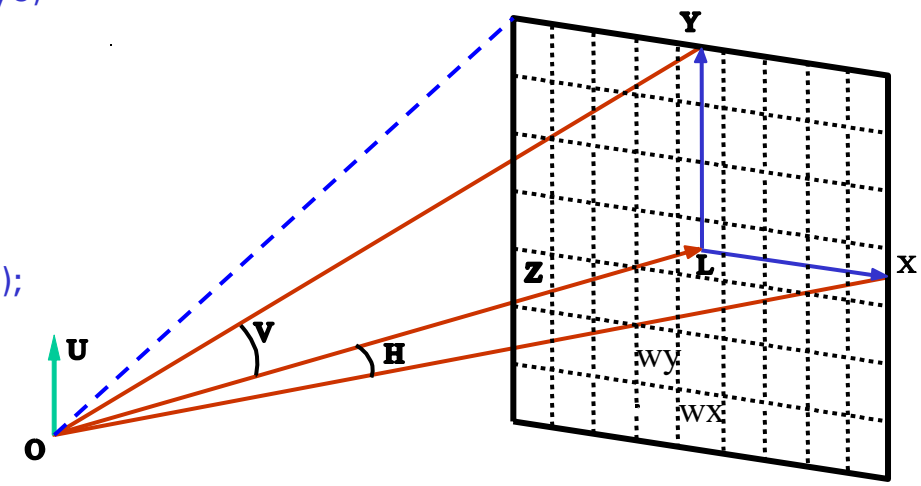


1. Ray tracing

1.2. Algoritmo.

- Función para calcular las direcciones de los rayos que se lanzan a la escena

```
Viewing  (int sizex, int sizey, float hfo, float vfo,
          Punto3D p_vision, Punto3D p_obser, Punto3D v_arriba,
          Punto3D *wx, Punto3D *wy, Punto3D *primer_rayo)
{
int  i, j;
Punto3D vista;
float distancia;

vista=p_vision-p_obser;
distancia=sqrt (vista.x*vista.x+vista.y*vista.y+vista.z*vista.z);
Normalizar (&vista);
Producto_Vectorial (vista, v_arriba, wx);
Producto_Vectorial(*wx, vista, wy);
distancia=2.0*distancia;

*wx=distancia*tan(hfo*RADGRA)/sizex;
*wy=distancia*tan(yfo*RADGRA)/sizey;
*primer_rayo=p_vision-p_obser;
primer_rayo->x= primer_rayo->x + (sizey*wy->x-sizex*wx->x)/2.0;
primer_rayo->y= primer_rayo->x + (sizey*wy->y-sizex*wx.y)/2.0;
primer_rayo->z= primer_rayo->z + (sizey*wy->z-sizex*wx_<z)/2.0;
}
```



1. Ray tracing

1.2. Algoritmo.

- Intersecciones. En ray tracing no solo se usan caras planas
  - Ejemplo: objeto esfera

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = r^2$$

$$x = x_o + x_d t \qquad y = y_o + y_d t \qquad z = z_o + z_d t$$

$$t_{\pm} = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

$$A = x_d^2 + y_d^2 + z_d^2$$
$$B = 2\left[x_d(x_o - c_x) + y_d(y_o - c_y) + z_d(z_o - c_z)\right]$$
$$C = (x_o - c_x)^2 + (y_o - c_y)^2 + (z_o - c_z)^2 - r^2$$

1. Ray tracing

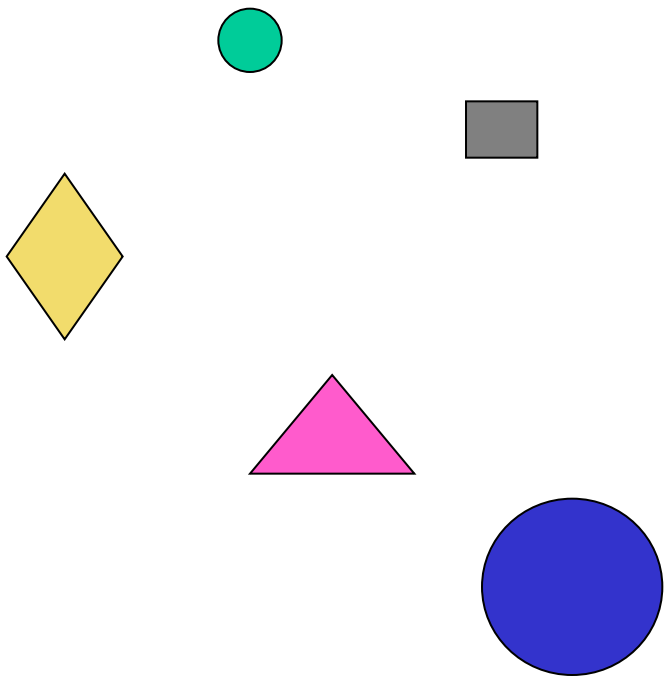
1.2. Algoritmo.

- Referencias sobre Intersecciones.
  - A. Glassner. “Introduction to Ray Tracing”. Morgan Kaufmann 1989 (octava impresión 2000)
  - W. Stürzlinger. “Ray Tracing Triangular Trimmed Free-Form Surfaces”. IEEE Trans. on Visualization and Computer Graphics, vol. 4, 1998 (NURBS)
  - C. Benthin et al. “Interactive Ray Tracing of Free-Form Surfaces”. Proceedings AFRIGRAPH 2004 (Superficies bicúbicas)
  - I. Wald and H. Seidel. “Interactive Ray Tracing of Point-based Models”. Symposium on Point Based Graphics 2005 (Puntos)
  - J. Kajiya. “New Techniques for Ray Tracing Procedurally Defined Objects”. Computer Graphics, vol., 1983 (Fractales)

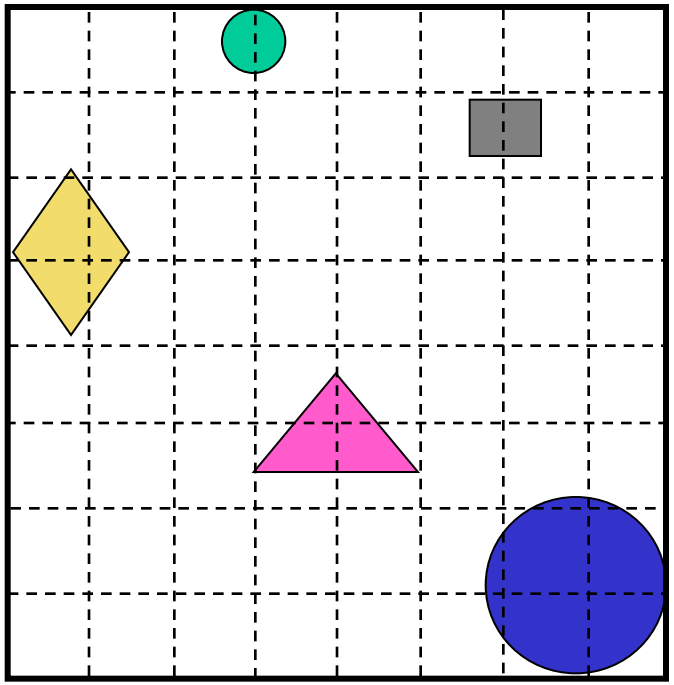
1. Ray tracing

1.2. Algoritmo (técnicas de optimización)

- División uniforme del espacio (vóxeles)



Escena inicial

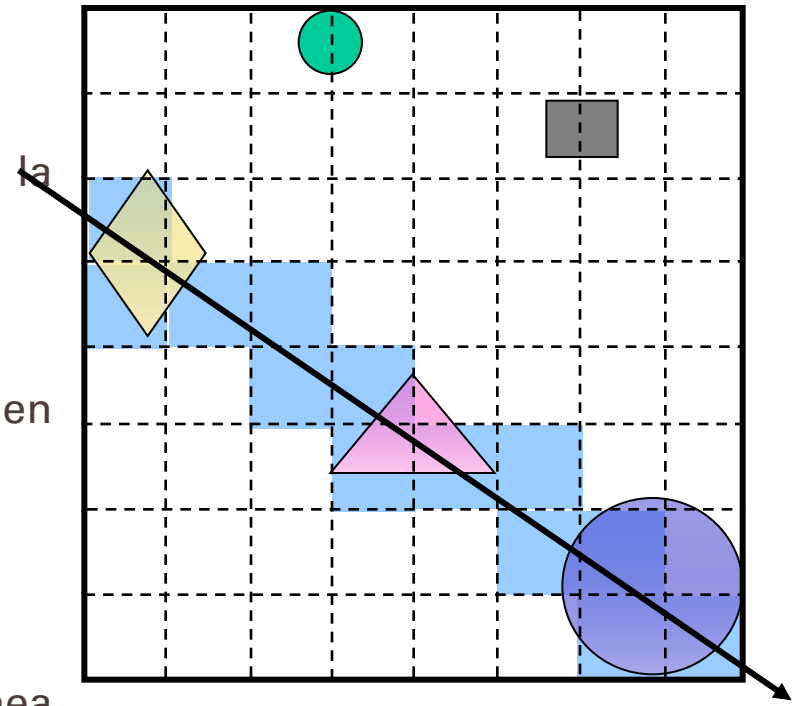


Escena preprocesada

1. Ray tracing

1.2. Algoritmo (técnicas de optimización)

- División uniforme del espacio (vóxeles)
  - Preprocesado
    - Calcular el volumen frontera de toda la escena
    - Determinar la resolución
    - Calcular la lista de objetos contenidos en todo o en parte en cada voxel
  - Recorrido de la escena
    - Se usa un algoritmo rasterización de línea en 3D y 6-conectado

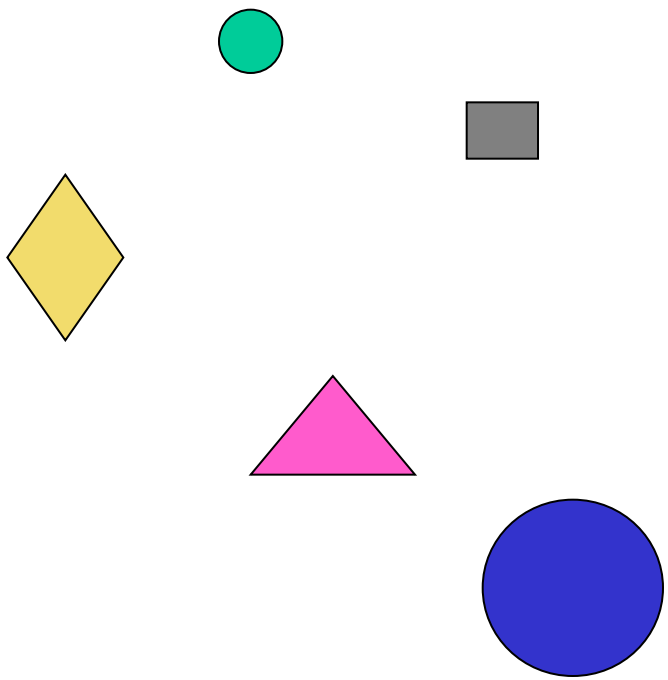


Recorrido de un rayo

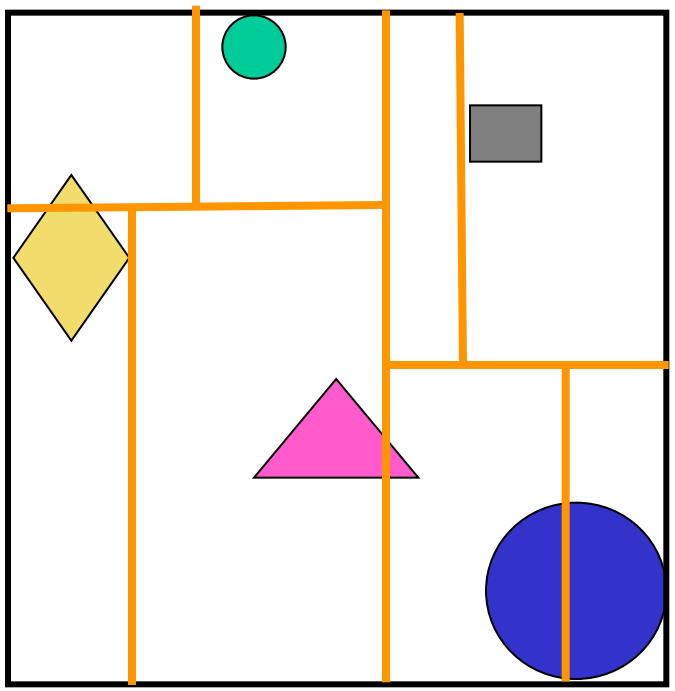
1. Ray tracing

1.2. Algoritmo (técnicas de optimización)

- Partición binaria del espacio con kd-tree



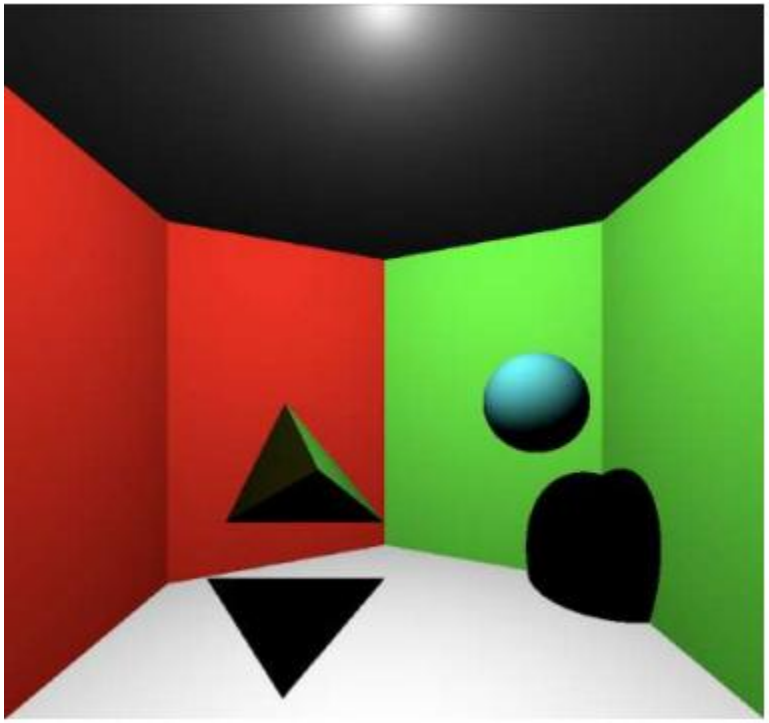
Escena inicial



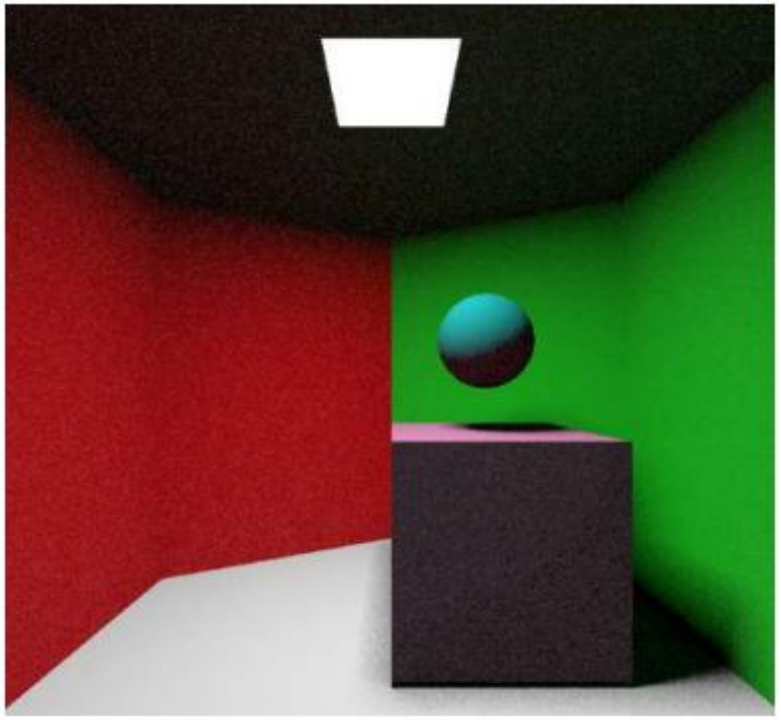
Escena preprocesada

1. Ray tracing

1.3. Ray tracing basado en Monte Carlo (MCRT) (1986)



Ray tracing clásico (Whitted)



Ray tracing Monte Carlo

Fuente: J. Forslund y P. Arnesson, “Monte Carlo Ray Tracer” , 2019

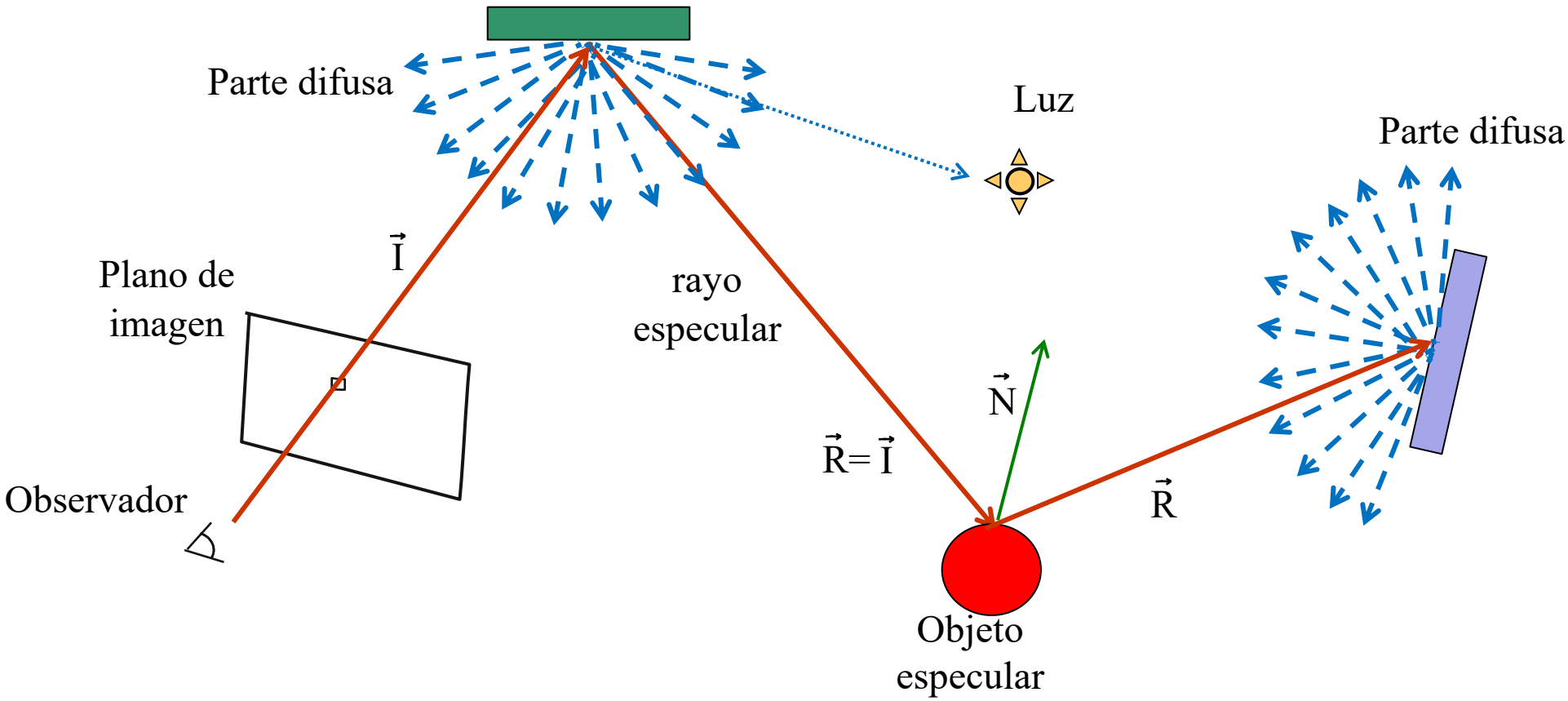
1. Ray tracing

1.3. Ray tracing basado en Monte Carlo

- En objetos con superficies que no sean perfectamente especulares o transparentes se tienen numerosos rayos salientes o transmitidos para un punto de corte.
- Con un método de Montercarlo se selecciona estocásticamente un conjunto de rayos seguir después de una intersección.
- Los rayos se siguen hasta que alcanzan una fuente de luz o cumplen una condición de parada.
- Entre otros efectos se obtienen sombras suaves al emplear luces no puntuales

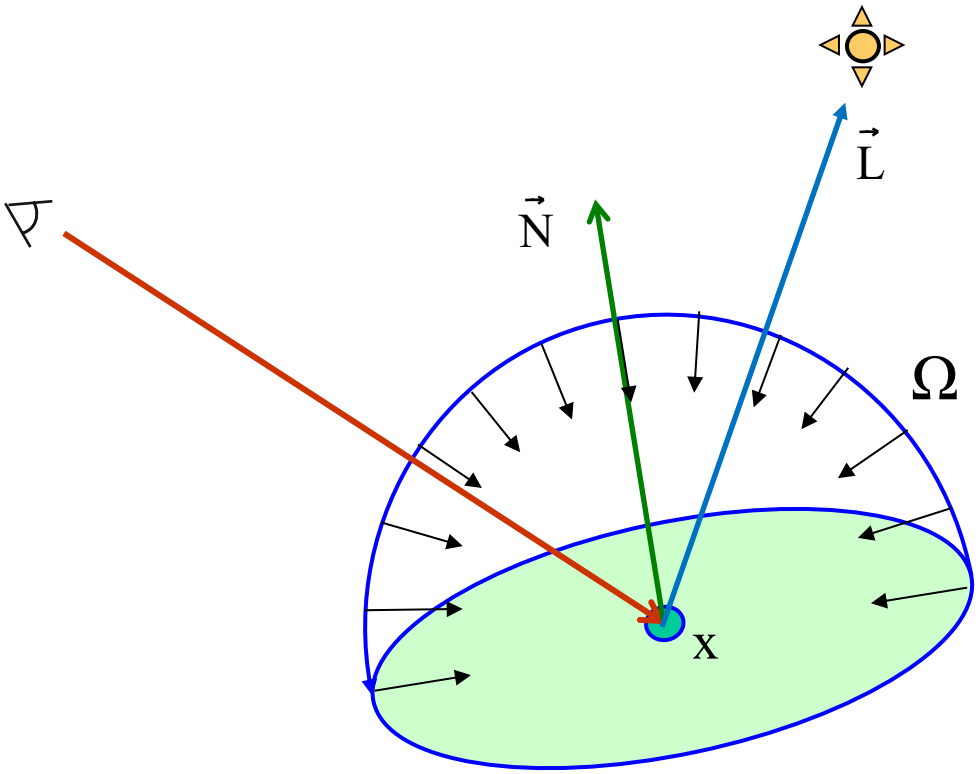
1. Ray tracing

1.3. Ray tracing basado en Monte Carlo



1. Ray tracing

1.3. Ray tracing basado en Monte Carlo

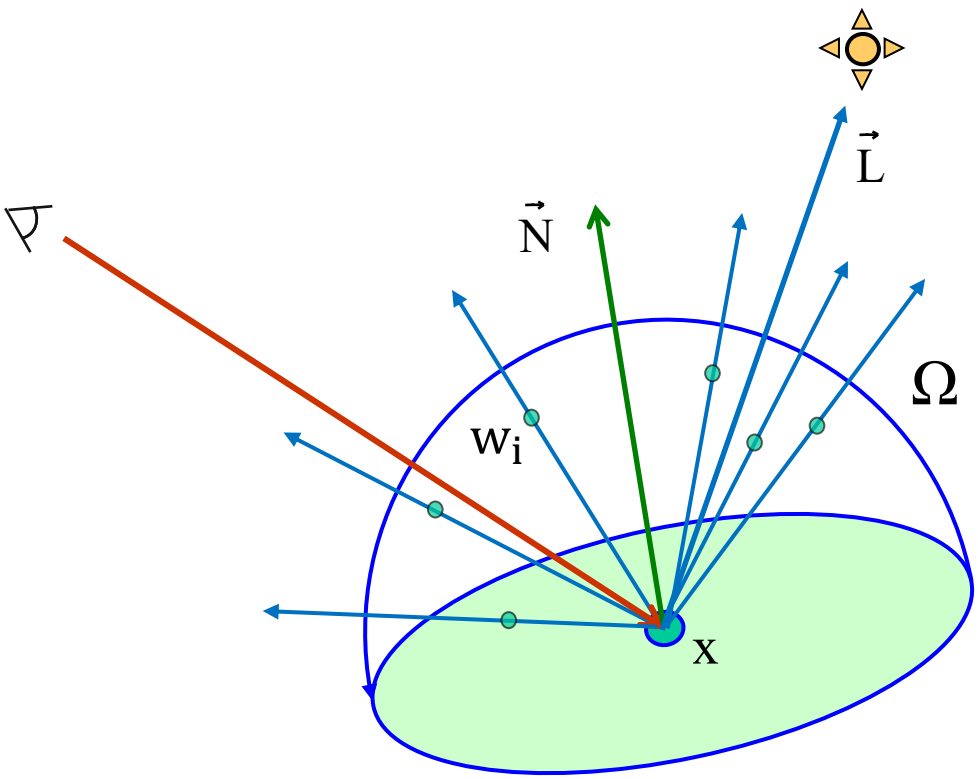


$$I_{difusa} = I_{directa} + I_{indirecta} =$$

$$I_L\left(\vec{N}\cdot\vec{L}\right) + I_{indirecta}$$

1. Ray tracing

1.3. Ray tracing basado en Monte Carlo



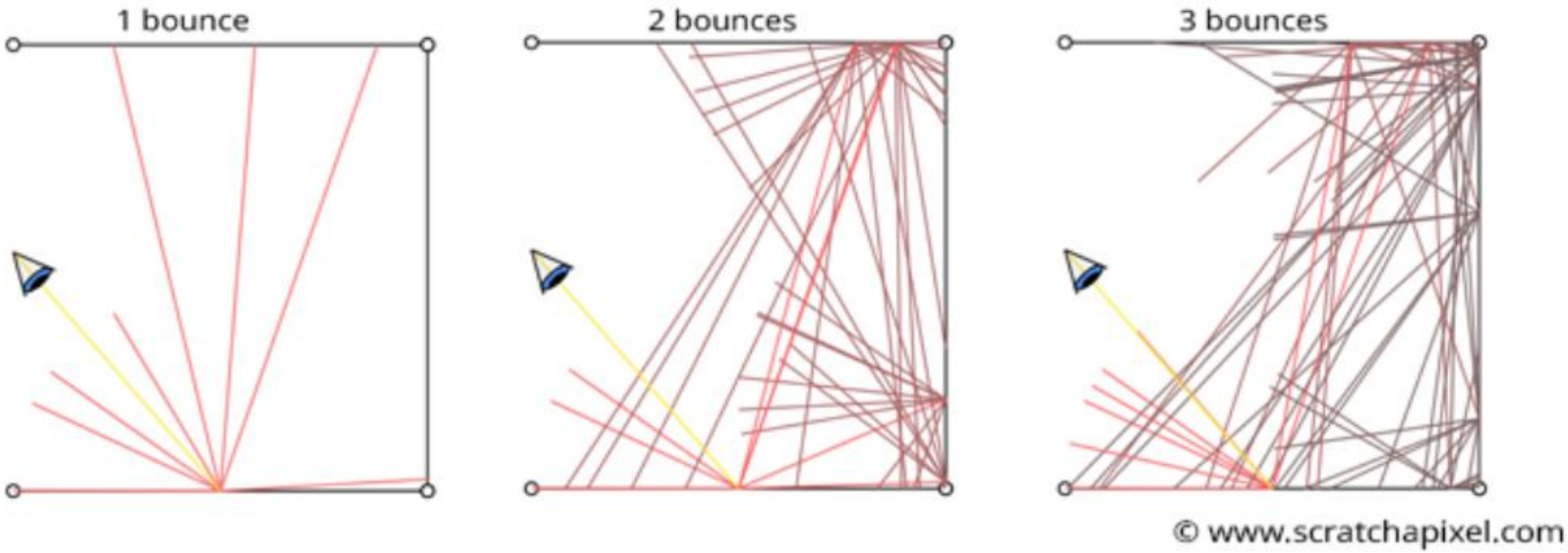
$$I_{indirecta} = \int_{\Omega} I_i(x, w_i)(\vec{N} \cdot w_i) dw_i$$

$$\approx \frac{1}{N} \sum_{k=1}^{N} I_i(x, w_i)(\vec{N} \cdot w_i)$$

w_i dirección del rayo

1. Ray tracing

1.3. Ray tracing basado en Monte Carlo



1. Ray tracing

1.4. Software

- POV-RAY (1991)
  - Un ejemplo de Ray Tracer de dominio público
  - Utiliza un lenguaje de descripción de escenas
  - Dirección: http://www.povray.org

1. Ray tracing

1.4. Software

- Lenguaje de descripción de escenas o SDL (Scene Description Language). Es un lenguaje declarativo que indica lo que hay en una escena (objetos (esferas, materiales, luces, cámaras)
- POV-RAY tiene su propio SDL
- **USD** (Universal Description Language) propuesto por Pixar es un estándar industrial, portable entre diversas aplicaciones como:
  - RenderMan
  - Unreal
  - Unity
  - Katana
  - Maya y 3ds Max

1. Ray tracing

1.4. Software (POV-RAY ejemplo)

```
#include "colors.inc"
#include "shapes.inc"

camera { location <0, 9, -60>  angle 20  look_at <-2, 4, 0>}

light_source { <30, 40, -100> color White }

background { color SkyBlue }

#declare P = polygon {  12,
  <0, 0>, <0, 6>, <4, 6>, <4, 3>, <1, 3>, <1, 0>, <0, 0>,
  <1, 4>, <1, 5>, <3, 5>, <3, 4>, <1, 4>
}

#declare O = polygon {  10,
  <0, 0, -3>, <0, 6, -3>, <4, 6, -3>, <4, 0, -3>, <0, 0, -3>,
  <1, 1, -3>, <1, 5, -3>, <3, 5, -3>, <3, 1, -3>, <1, 1, -3> }

#declare V = polygon { 8,
  <1, 0>, <0, 6>, <1, 6>, <2, 1>, <3, 6>, <4, 6>, <3, 0>, <1, 0>}
```

1. Ray tracing

1.4. Software (POV-RAY ejemplo. Continuación)

```
union {
  object { O translate 0*x }
  object { V translate 5*x }
  pigment { colour rgb<0.9,0.7,0.4> }
  finish { ambient 0.1  diffuse 0.2 reflection 1}
  translate -3*x
}

object {P  translate -8.2*x
        pigment { colour rgb<0.9,0.7,0.4>}
        finish { ambient 0.1  diffuse 0.7}}

plane { y, 0
  pigment { color rgb<0.3,1,0> }
}
```

1. Ray tracing

1.4. Software (POV-RAY ejemplo. Continuación)

POV-Ray - C:\Users\alopez\Documents\POV-Ray\v3.7\scenes\objects\polygon.pov [Idle]

File Edit Search Text Editor Insert Render Options Tools Window Help

New Open Save Close Queue Hide Ini Sel-Run Run Pause Tray

State: Idle
Frame: N/A

[640x480, No AA] POV-Win Scene POV Site

Messages polygon.pov textures.inc

```
  angle 20
  look_at <-2, 4, 0>
}

light_source { <30, 40, -100> color White }

background { color SkyBlue }

#declare P =
polygon {
  12,
  <0, 0>, <0, 6>, <4, 6>, <4, 3>, <1, 3>, <1, 0>, <0, 0>,
  <1, 4>, <1, 5>, <3, 5>, <3, 4>, <1, 4>
}

#declare O =
polygon {
  10,
  <0, 0, -3>, <0, 6, -3>, <4, 6, -3>, <4, 0, -3>, <0, 0, -3>,
  <1, 1, -3>, <1, 5, -3>, <3, 5, -3>, <3, 1, -3>, <1, 1, -3>
}

#declare V =
polygon {
  8,
  <1, 0>, <0, 6>, <1, 6>, <2, 1>, <3, 6>, <4, 6>, <3, 0>, <1, 0>
}

union {
  object { O translate 0*x }
  object { V translate 5*x }
  pigment { colour rgb<0.9,0.7,0.4> }
  finish { ambient 0.1  diffuse 0.2 reflection 1}
  translate -3*x
}

object {P  translate -8.2*x
           pigment { colour rgb<0.9,0.7,0.4>}
           finish { ambient 0.1  diffuse 0.7}}

plane { y, 0
  texture{ pigment{ Jade }
           finish{ diffuse 1.0 }
         }
}
```

281,125 [0.440,0.261]

1. Ray tracing

1.4. Software

- Mental ray (1986), NVIDIA (2007)
  - Utilizado en 3ds max hasta 2017
  - Permitía simular luces fotométricas
  - Defenestrado por NVIDIA en 2018

1. Ray tracing

1.4. Software

- Arnold (Solid Angle)
  - Utilizado desde 2018 en 3D Studio Max. En 2016 Autodesk compró Arnold
  - Ray tracing basado en Monte Carlo
  - Dirección: https://www.arnoldrenderer.com/

1. Ray tracing

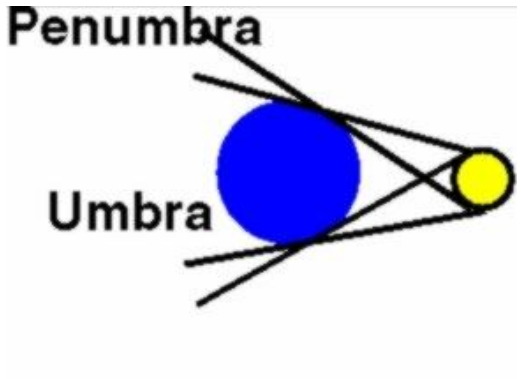
1.4. Software

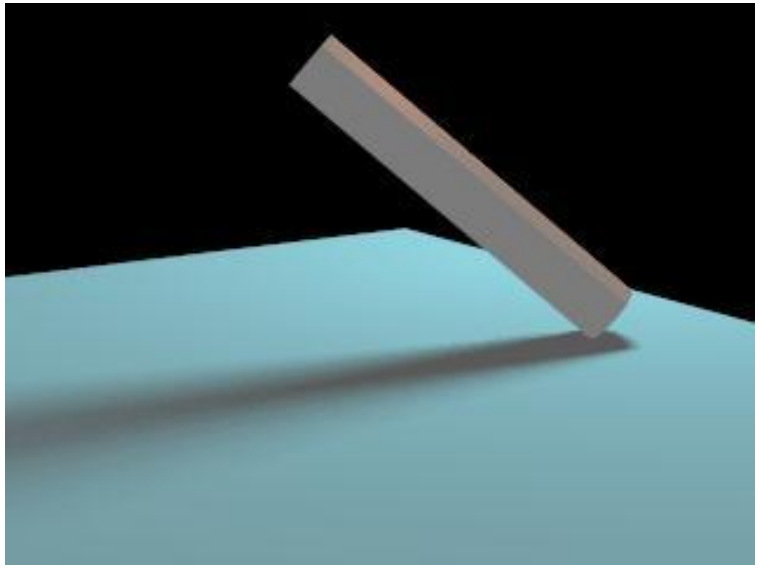
- Otros
  - Yafray (usado antes en Blender) https://github.com/YafaRay/libYafaRay
  - CYcles (integrado en Blender 4.3) https://www.cycles-renderer.org/
  - Radiance (un clásico) http://radsite.lbl.gov/radiance/
  - 3Delight https://www.3delight.com/
  - Appleseed http://appleseedhq.net/

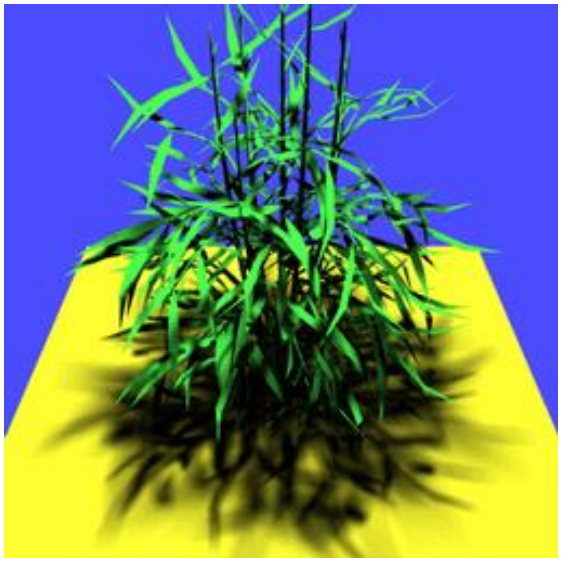
1.5. Ray tracing (efectos)

- Penumbra (Soft Shadows)

  P. Shirley. "Realistic Ray Tracing". A. K. Peters LTD, 2000





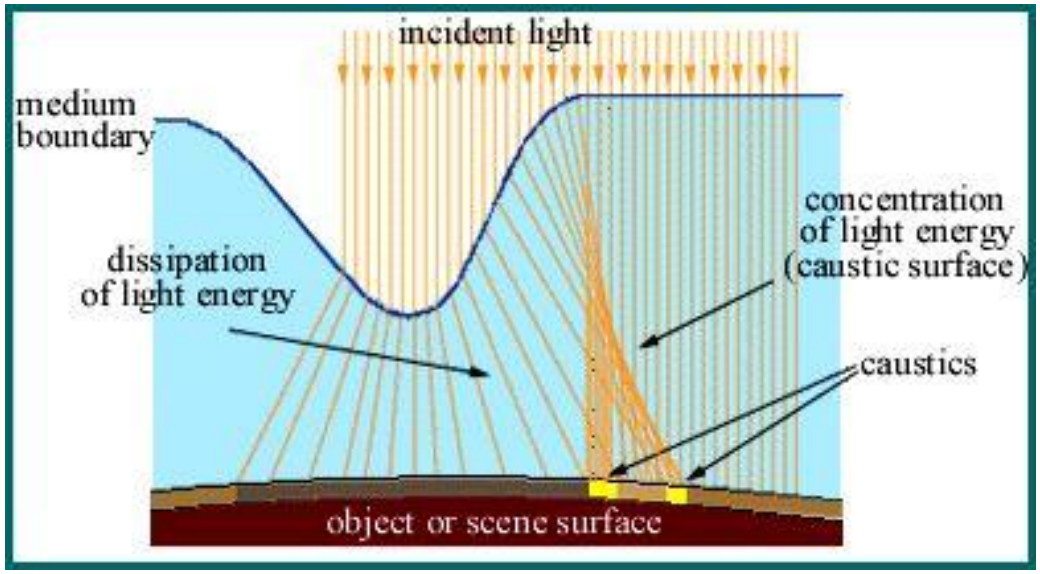


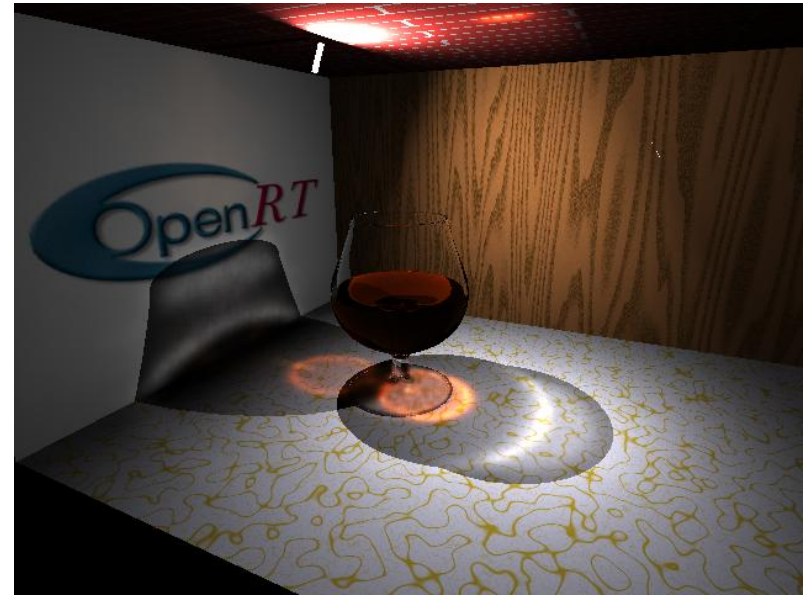
1.5. Ray tracing (efectos)

- Reflexión cáustica

  J. Arvo, J. 1986. "Backward ray tracing". Proceedings of SIGGRAPH 1986

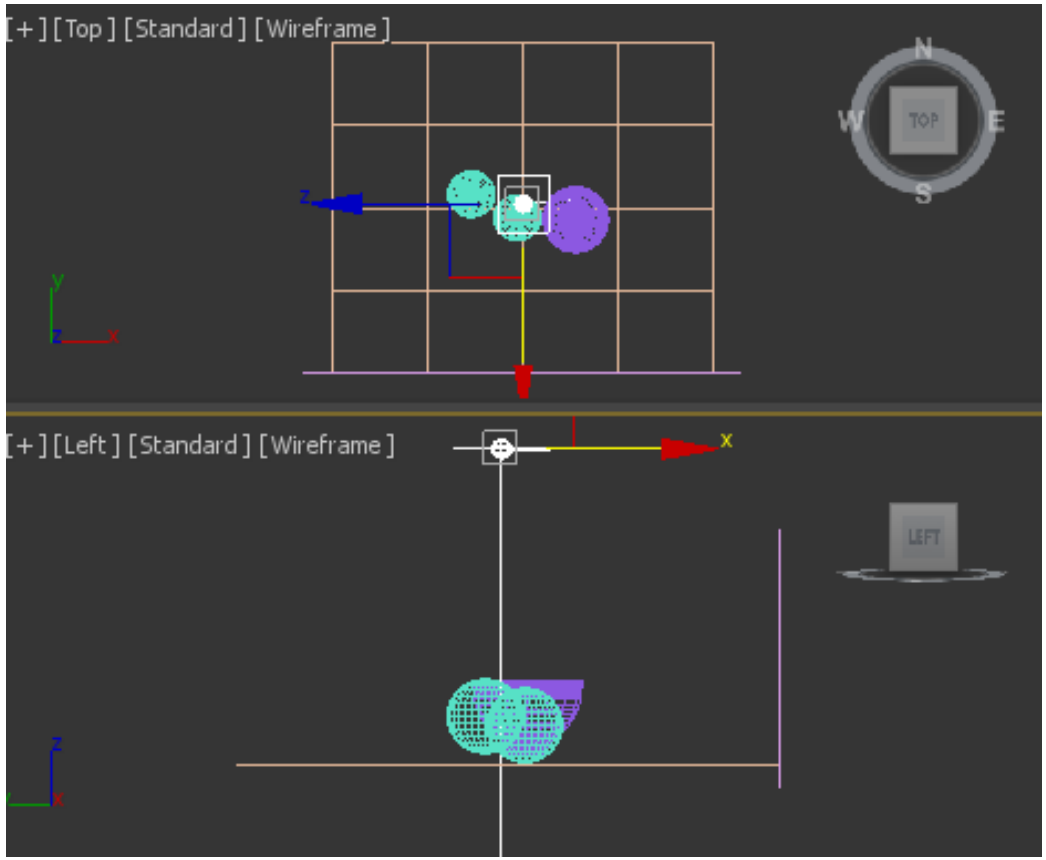
  J. Günther, I. Wald, P. Slusallek. "Realtime Caustics using Distributed Photon Mapping". Eurographics Symposium on Rendering 2004

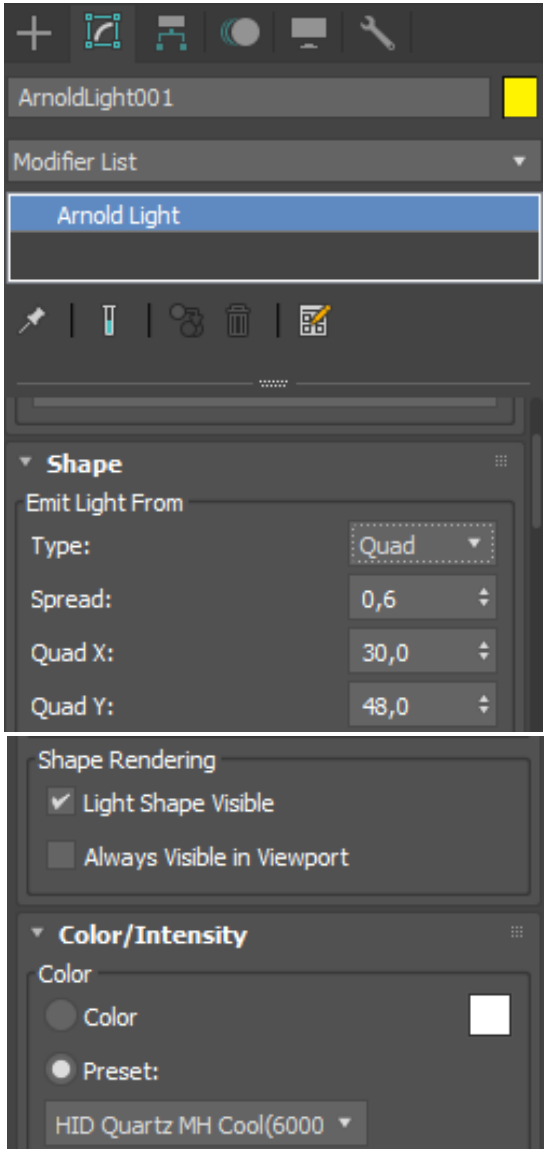




1.5. Ray tracing (efectos)

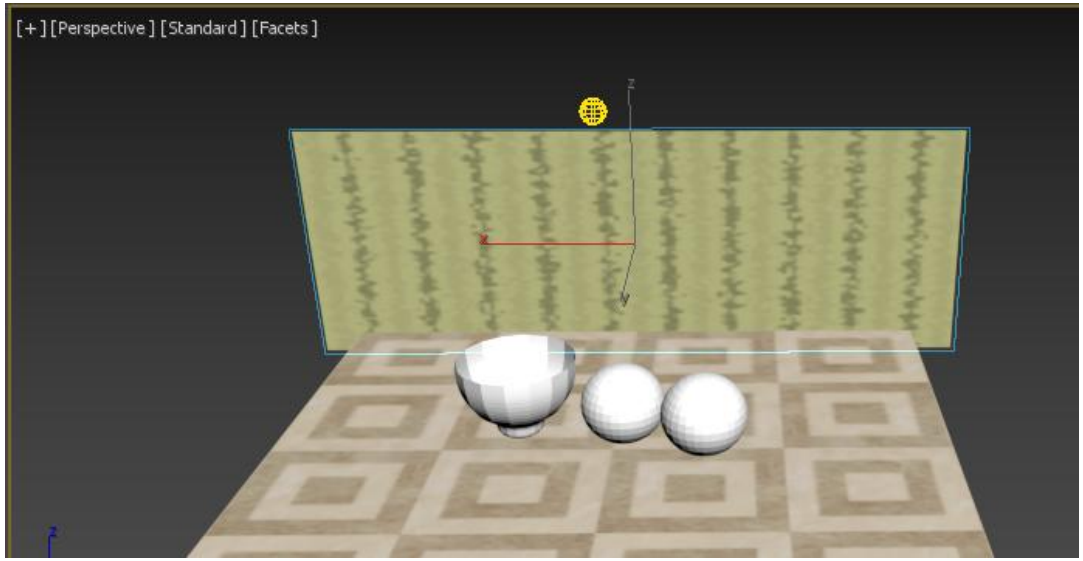
- Reflexión cáustica con Autodesk 3ds Max
  - Luces extensas tratadas como objetos
  - Usar como render Arnold

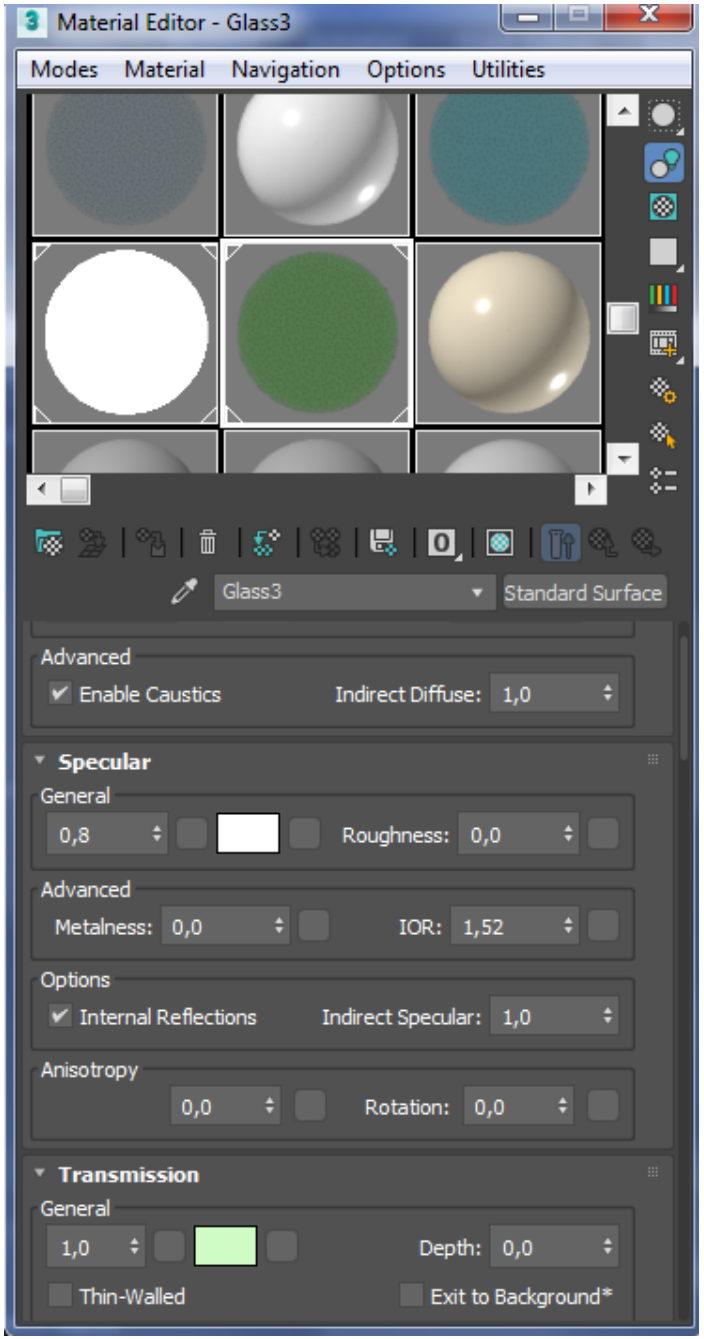




1.5. Ray tracing (efectos)

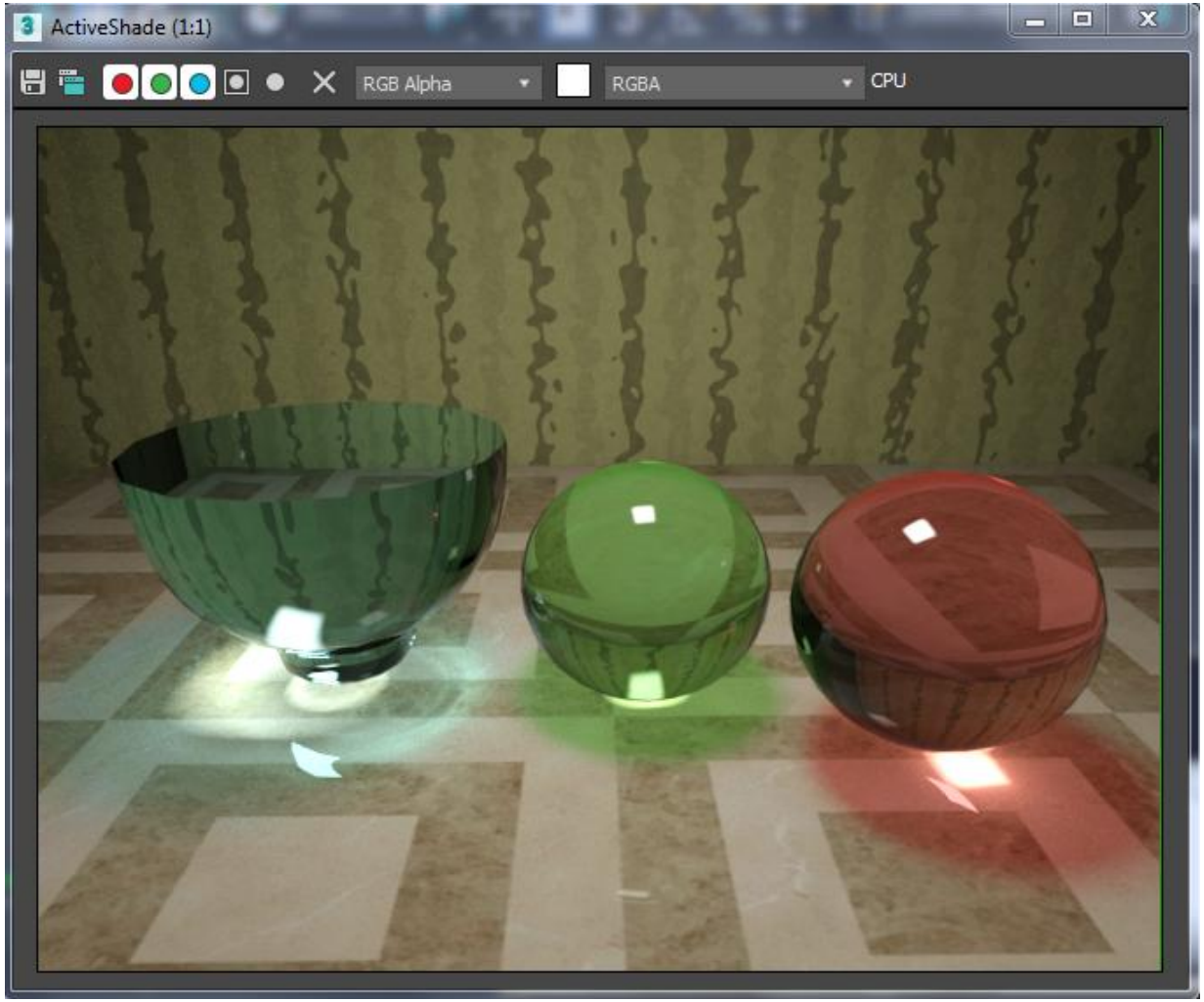
- Reflexión cáustica con Autodesk 3ds Max
  - Objetos con material transparente, especular y con iluminación caústica activada





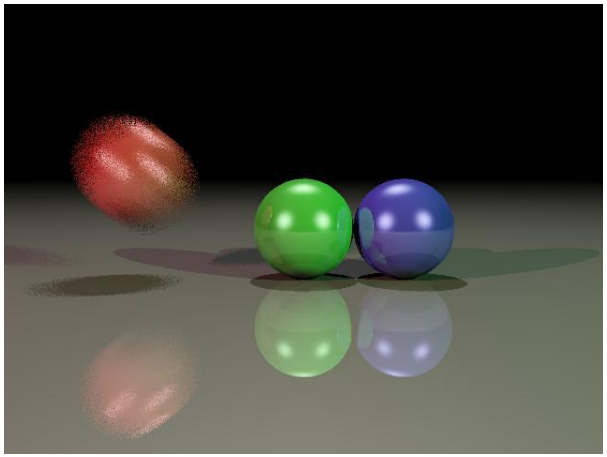
1.5. Ray tracing (efectos)

Reflexión cáustica con Autodesk 3ds Max

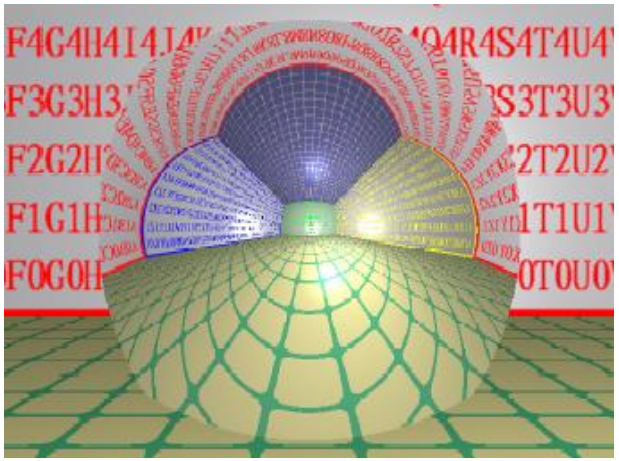


1.5. Ray tracing (efectos)

- Motion Blur



- Relatividad espacial (https://www.techfreaq.de/grraytracing.html)



2. Radiosidad

- También se pretende simular todas las reflexiones de la luz en una escena (iluminación local frente a iluminación global)

2. Radiosidad

- Introducida por Goral et al. (1984)
- Solo calcula la iluminación difusa entre superficies
- Requiere preparar la escena (discretización en parches)
- Se combina con Ray Tracing para la iluminación especular
- Independencia de la cámara
- Hoy se prefiere el Ray Tracing de Montecarlo al abarcar directamente todas las interacciones de la luz con superficies, ser paralelizable y más práctico en escenas dinámicas
- Empleado para arquitectura en diseño de iluminación de estancias (https://www.dialux.com/).

2. Radiosidad

- Ecuación de rendering
- Relaciona la energia total emitida de una superficie con la energía emitida debida a que la superficie sea un emisor y la energía emitida debida a la energía recibida por otras superficies.

$$L(\omega_0) = L_e(\omega_0) + \int_{\omega_i} \rho(\omega_0, \omega_i) L(\omega_i) \cos\phi_i \, d\omega_i$$

Radiancia total o Energía total emitida

Radiancia o energía emitida debida que la superficie sea emisora (fuente deluz)

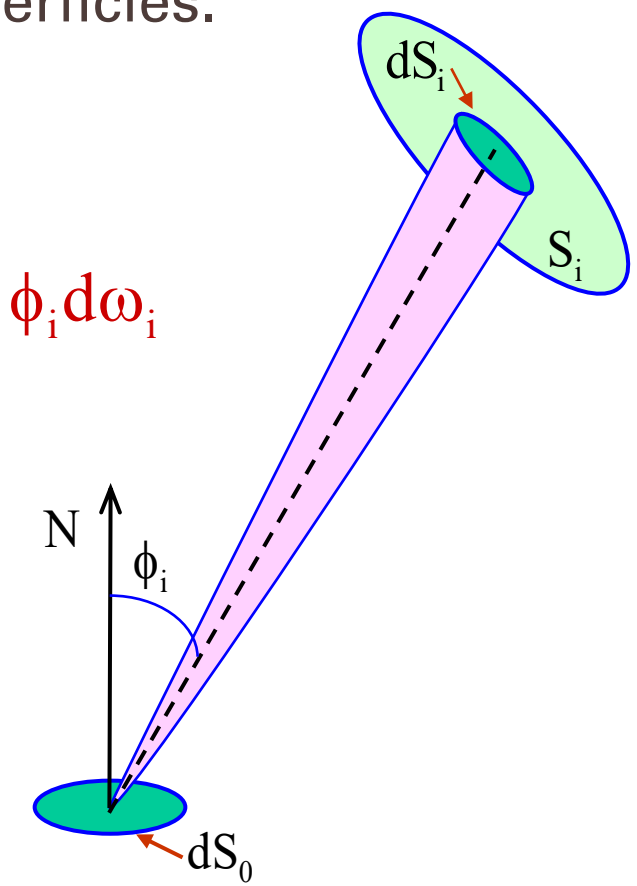
Función de distribución de reflectancia bidireccional (BRDF)

Radiancia o energía emitida debida que la energía recibida desde todas las superficies de una escena

2. Radiosidad

- Ecuación de rendering
- La función de distribución de reflectancia bidireccional (bidirectional reflectance distribibution function o BRDF) depende de las propiedades de las superficies.

$$L(\omega_0) = L_e(\omega_0) + \int_{\omega_i} \rho(\omega_0, \omega_i) L(\omega_i) \cos\phi_i d\omega_i$$



2. Radiosidad

- Es una solución de elementos finitos a la ecuación de rendering
- La escena se discretiza el áreas llamadas pacthes

$$B_i A_i = E_i A_i + \rho_i \sum_{j=1}^{n} B_j A_j \, F_{A_j A_i}$$

  - Siendo
    B_i la radiosidad del área i (A_j)
    E_i la energía emitida desde A_j
    ρ_i el coeficiente de reflectividad de A_j
    F_{dAidAj} el factor de forma, que representa la fracción de energía que partiendo de la superficie A_j alcanza la superficie A_i

2. Radiosidad

- Relación de reciprocidad entre factores de forma:

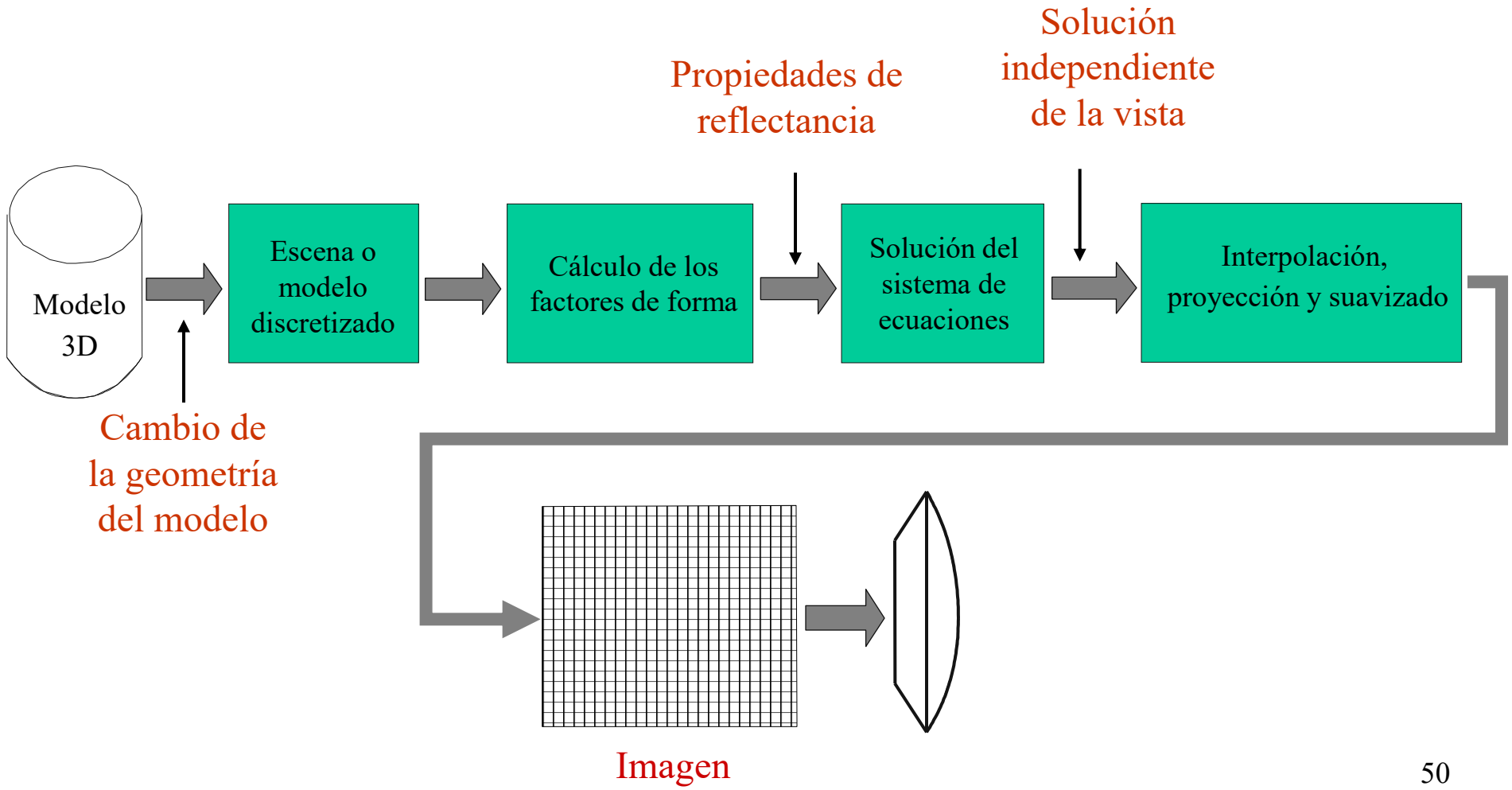
$$F_{A_iA_j}A_i = F_{A_jA_i}A_j \quad \rightarrow \quad F_{A_iA_j} = F_{A_jA_i}\frac{A_j}{A_i} \quad \rightarrow \quad B_i = E_i + \rho_i \sum_{j=1}^{n} B_j F_{A_iA_j}$$

- Sistema de ecuaciones para las B_i incógnitas:

$$\begin{pmatrix} B_1 \\ B_2 \\ . \\ . \\ B_n \end{pmatrix} = \begin{pmatrix} E_1 \\ E_2 \\ . \\ . \\ E_n \end{pmatrix} + \begin{pmatrix} \rho_1 F_{A_1A_1} & \rho_1 F_{A_1A_2} & . & . & \rho_1 F_{A_1A_n} \\ \rho_2 F_{A_2A_1} & \rho_2 F_{A_2A_2} & . & . & \rho_2 F_{A_2A_n} \\ . & . & . & . & . \\ . & . & . & . & . \\ \rho_n F_{A_nA_1} & \rho_n F_{A_nA_2} & . & . & \rho_n F_{A_nA_n} \end{pmatrix} \begin{pmatrix} B_1 \\ B_2 \\ . \\ . \\ B_n \end{pmatrix}$$

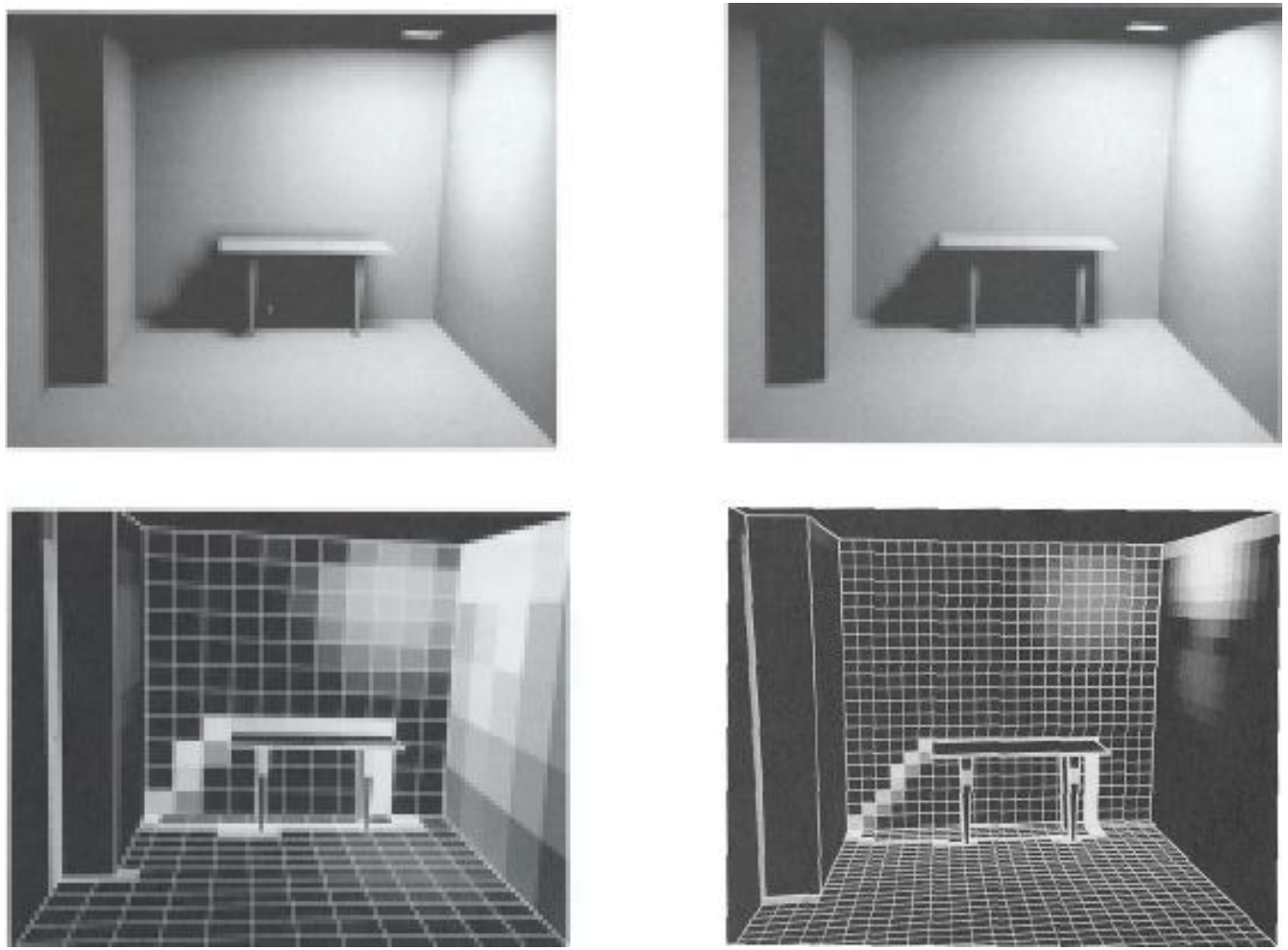
2. Radiosidad

- Etapas en un algoritmo de radiosidad



2. Radiosidad

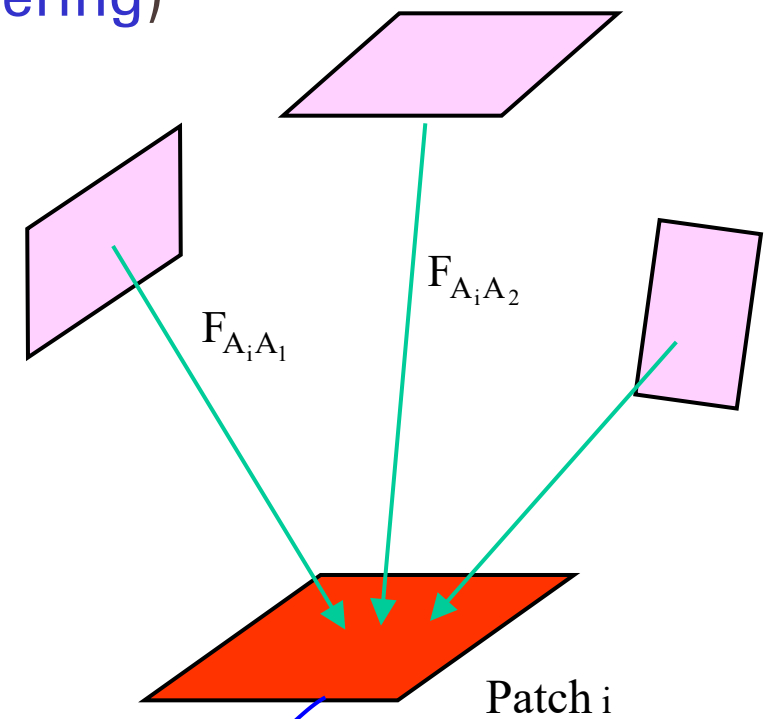
- Discretización de la escena



2. Radiosidad

- Solución del sistema de ecuaciones
  - Relajación de Gauss-Seidel (gathering)

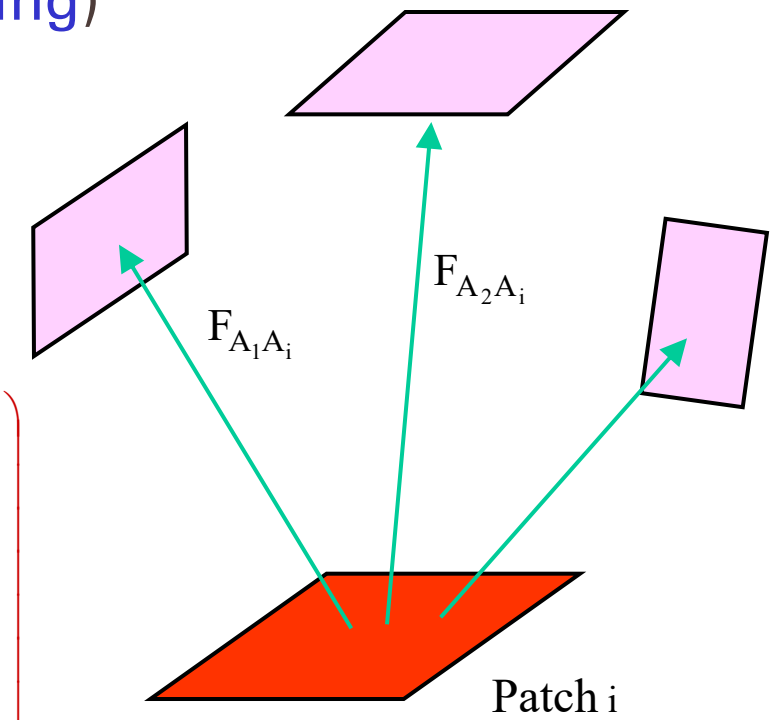
$$\begin{pmatrix} B_1^t \\ B_2^t \\ \cdot \\ B_i^t \\ \cdot \\ B_n^{t-1} \end{pmatrix} = \begin{pmatrix} E_1 \\ E_2 \\ \cdot \\ E_i \\ \cdot \\ E_n \end{pmatrix} + \begin{pmatrix} \rho_1 F_{A_1A_1} & \rho_1 F_{A_1A_2} & \cdot & \cdot & \rho_1 F_{A_1A_n} \\ \rho_2 F_{A_2A_1} & \rho_2 F_{A_2A_2} & \cdot & \cdot & \rho_2 F_{A_2A_n} \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \rho_i F_{A_iA_1} & \rho_i F_{A_iA_2} & \cdot & \cdot & \rho_i F_{A_iA_n} \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \rho_n F_{A_nA_1} & \rho_n F_{A_nA_2} & \cdot & \cdot & \rho_n F_{A_nA_n} \end{pmatrix} \begin{pmatrix} B_1^t \\ B_2^t \\ \cdot \\ B_i^{t-1} \\ \cdot \\ B_n^{t-1} \end{pmatrix}$$



2. Radiosidad

- Solución del sistema de ecuaciones
  - Relajación de Southwell (shooting)

$$\begin{pmatrix} B_1^t \\ B_2^t \\ \cdot \\ B_i^t \\ \cdot \\ B_n^t \end{pmatrix} = \begin{pmatrix} B_1^{t-1} \\ B_2^{t-1} \\ \cdot \\ B_i^{t-1} \\ \cdot \\ B_n^{t-1} \end{pmatrix} + \begin{pmatrix} B_1^{t-1} \\ B_2^{t-1} \\ \cdot \\ B_i^{t-1} \\ \cdot \\ B_n^{t-1} \end{pmatrix} \begin{pmatrix} \rho_1 F_{A_1A_1} & \cdot & \rho_1 F_{A_1A_i} & \cdot & \rho_1 F_{A_1A_n} \\ \rho_2 F_{A_2A_1} & \cdot & \rho_2 F_{A_2A_i} & \cdot & \rho_2 F_{A_2A_n} \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \rho_i F_{A_iA_1} & \cdot & \rho_i F_{A_iA_i} & \cdot & \rho_i F_{A_iA_n} \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \rho_n F_{A_nA_1} & \cdot & \rho_n F_{A_nA_i} & \cdot & \rho_n F_{A_nAn} \end{pmatrix}$$

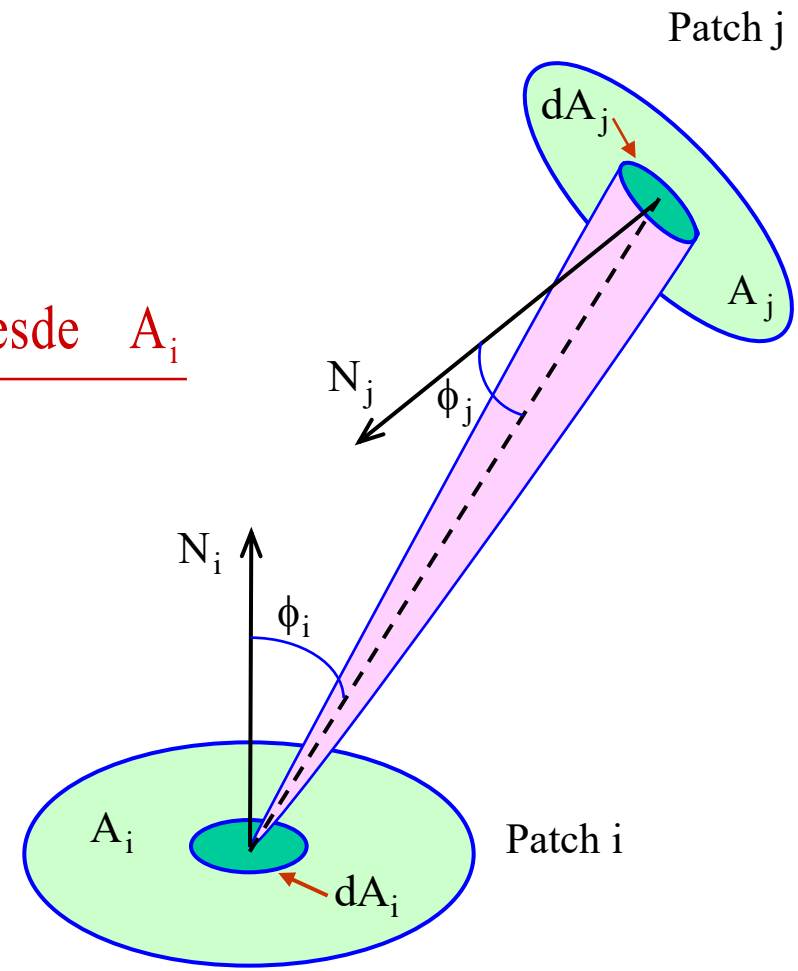


2. Radiosidad

- Cálculo de los factores de forma
  - Definición

$$F_{A_iA_j} = \frac{\text{Energía radiante que alcanza } A_j \text{ desde } A_i}{\text{Energía radiante total de } A_i}$$

$$F_{A_iA_j} = \frac{1}{A_i} \oint_{A_i} \oint_{A_j} \frac{\cos\phi_i \cos\phi_j}{\pi r^2} dA_j \, dA_i$$

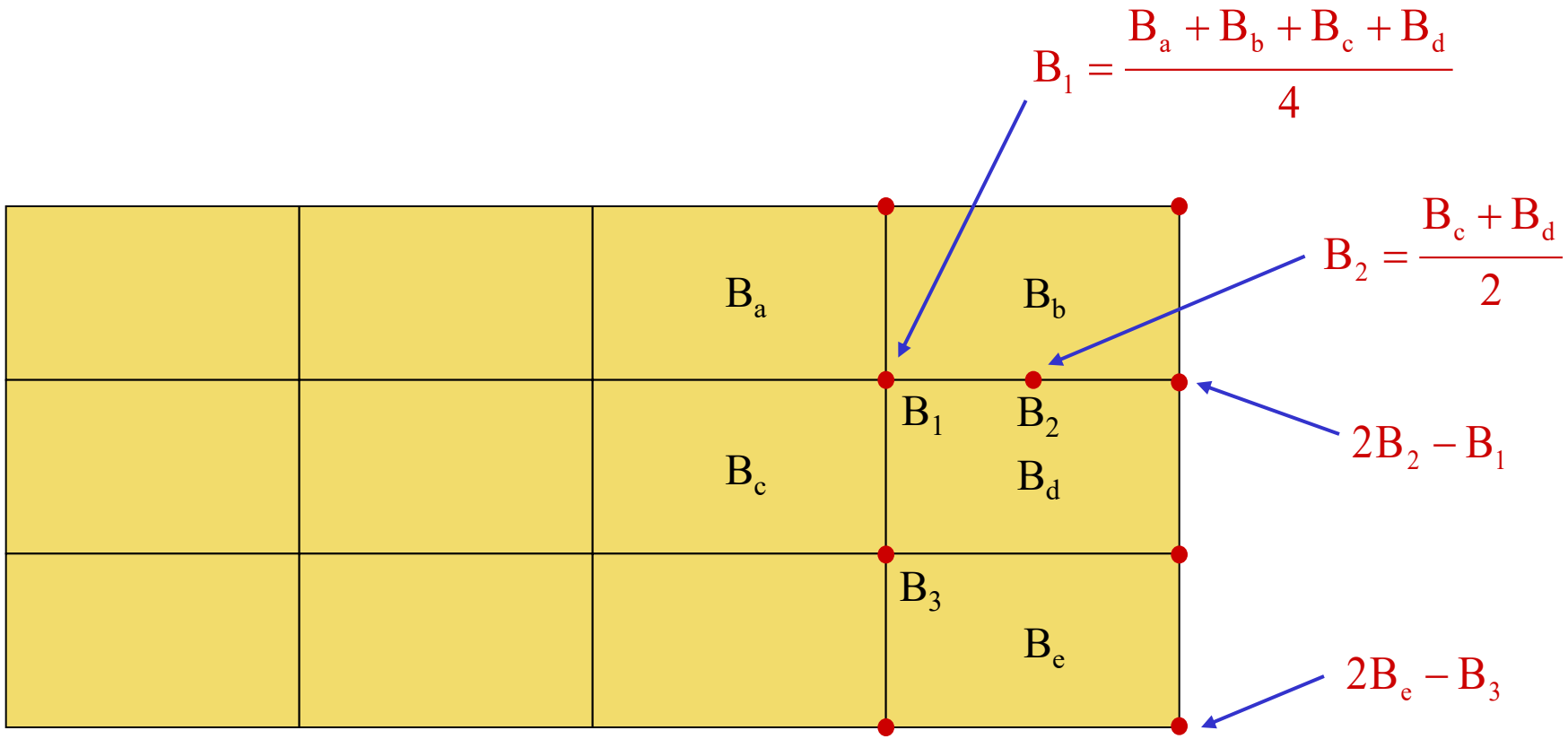


2. Radiosidad

- Interpolación
  - Se necesitan conocer las intensidades en los vértices de cada patch

2. Radiosidad

- Interpolación

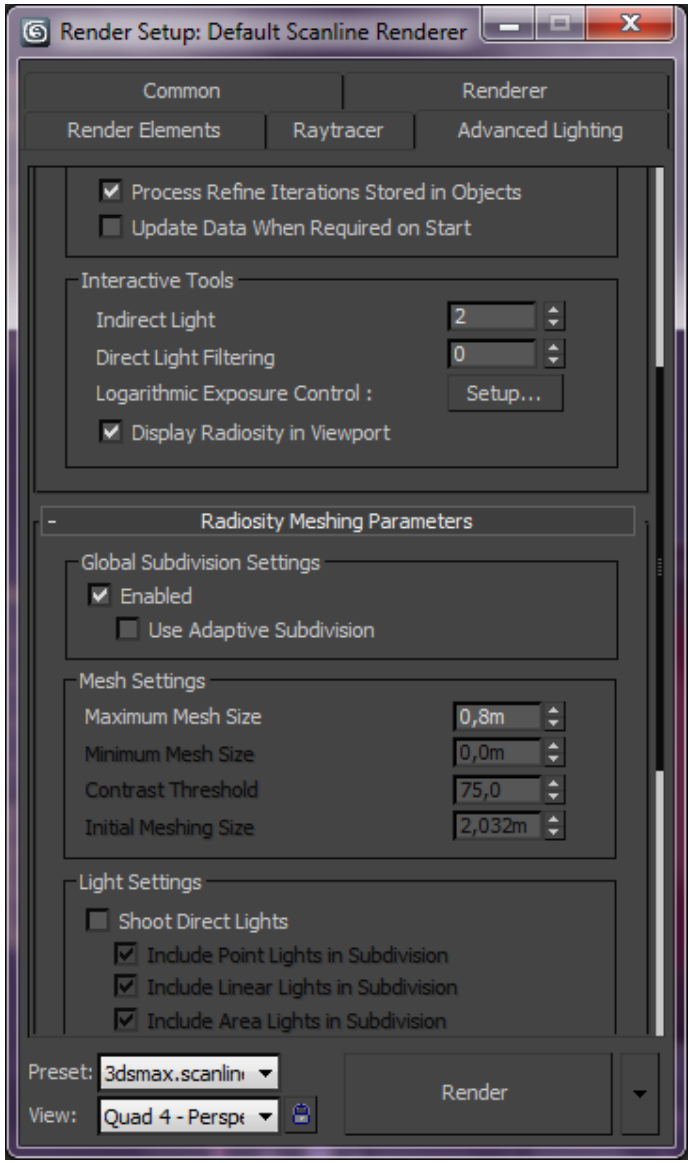


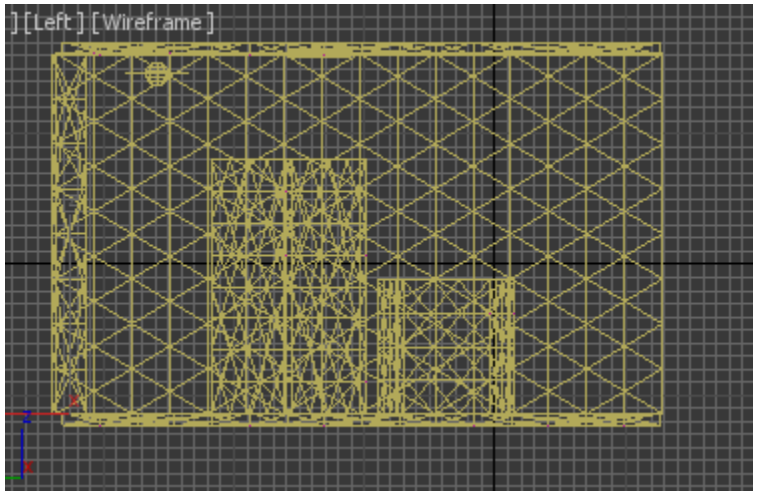
2. Radiosidad

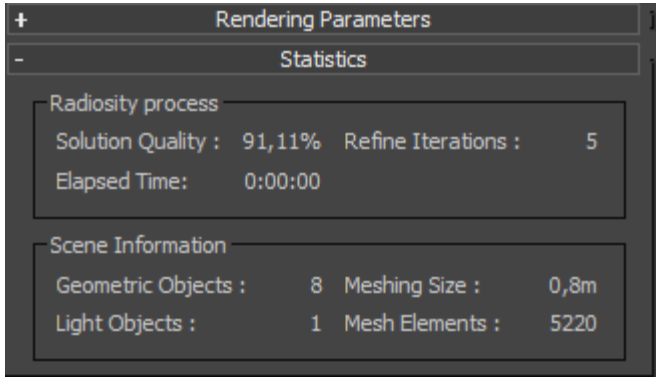
- Suavizado de intensidades

2. Radiosidad

- 3ds max

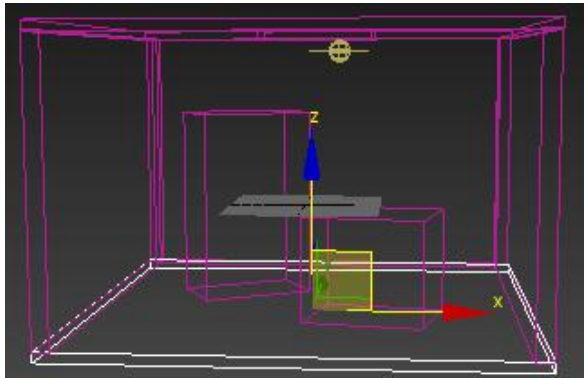


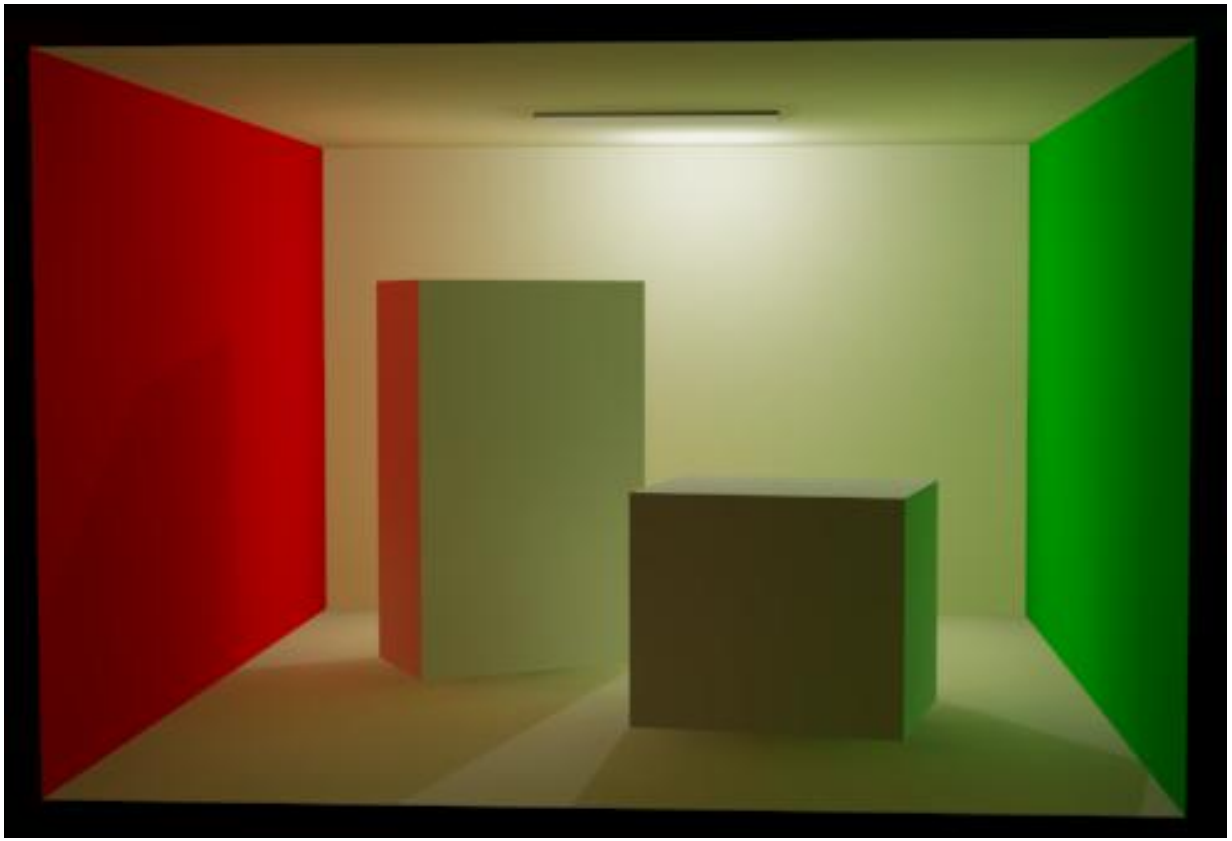




2. Radiosidad

- 3ds max





2. Radiosidad

- Combinando Radiosidad y Ray Tracing.

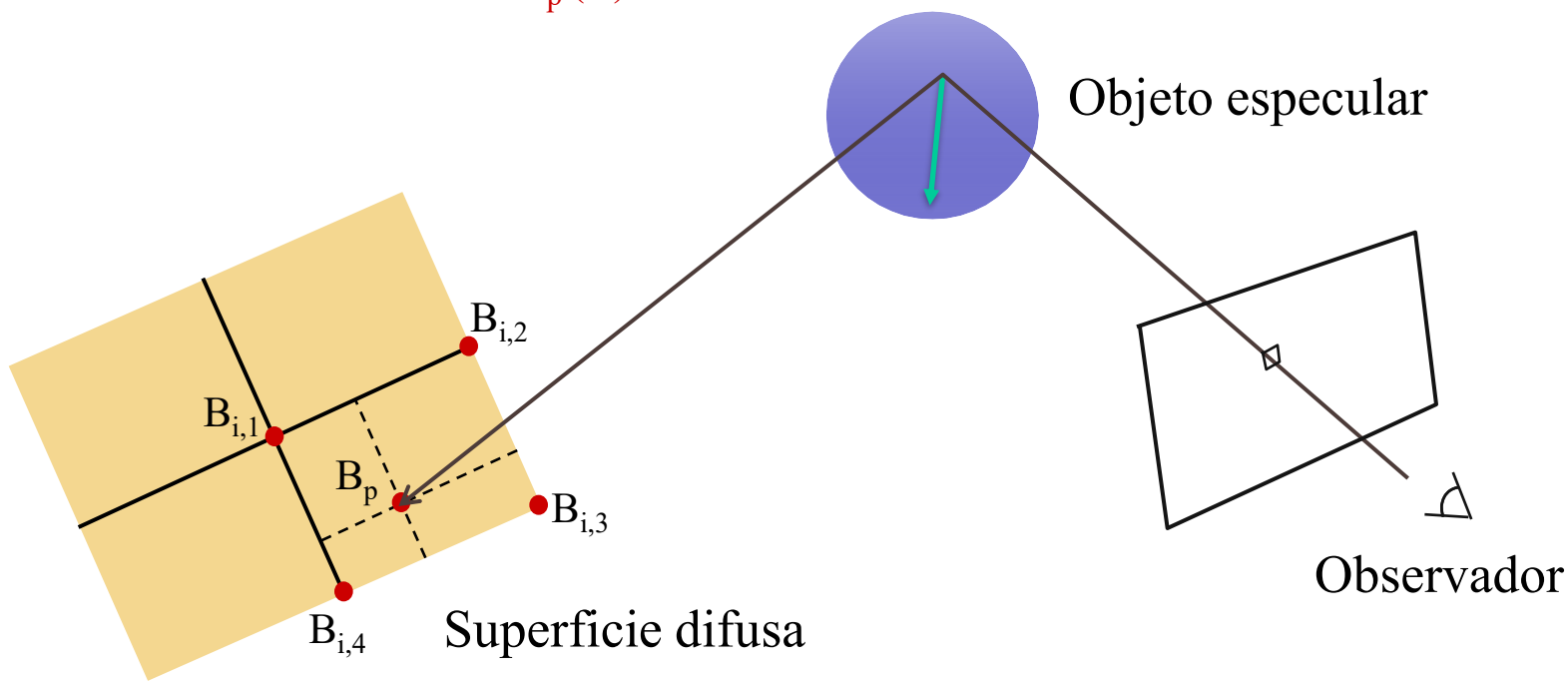
- Primera etapa: Radiosidad
  - Calcular factores de forma y la radiosidad

- Segunda etapa: Ray Tracing
  - Realizar un Ray Tracing tradicional
  - Sustituir las componentes ambientales y difusas de la iluminación local por la radiosidad

2. Radiosidad

- Combinando Radiosidad y Ray Tracing

$$I_\lambda = \underbrace{I_{a\lambda}K_{a\lambda} + \sum_i S_i I_{L(i)\lambda}\left(K_{d\lambda}\,(\vec{N}\,\vec{L}_i)_+\right.}_{=B_p(\lambda)} \left. + K_s\left(\vec{R}_{L(i)}\,\vec{V}\right)_+^n\right) + I_{r\lambda}K_s + I_{t\lambda}K_t$$



3. Photon Mapping

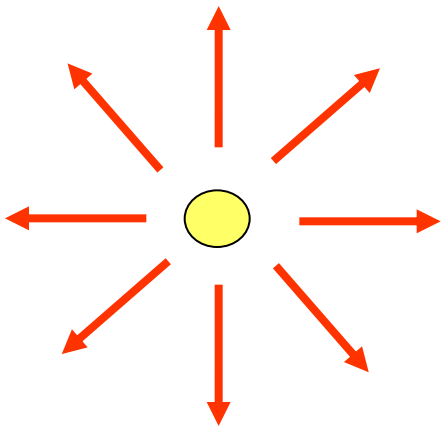
- Desarrollado por Jensen y Christensen en 1996. Puede simular caústica y medios participativos.

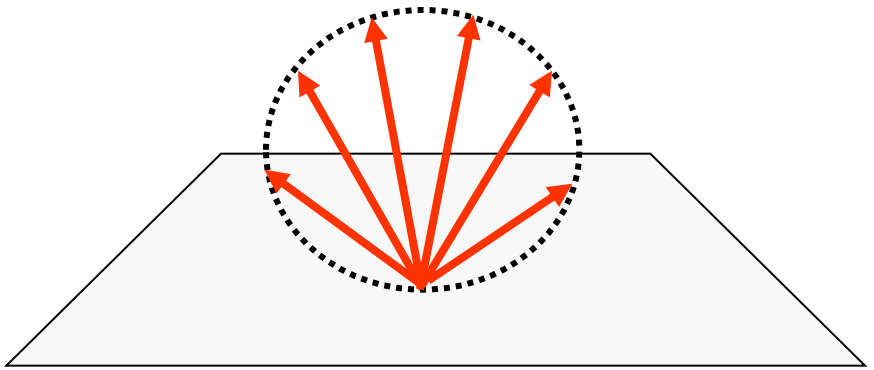
- Se basa en Monte Carlo ray tracing, pero con una solución más óptima

- Consta de dos pasos:
  - Photon tracing
    - Emisión de fotones
    - Seguimiento de fotones
    - Mapa de fotones
  - Síntesis de la imagen: calcular la radiancia L en un punto dado de una superficie

3. Photon Mapping

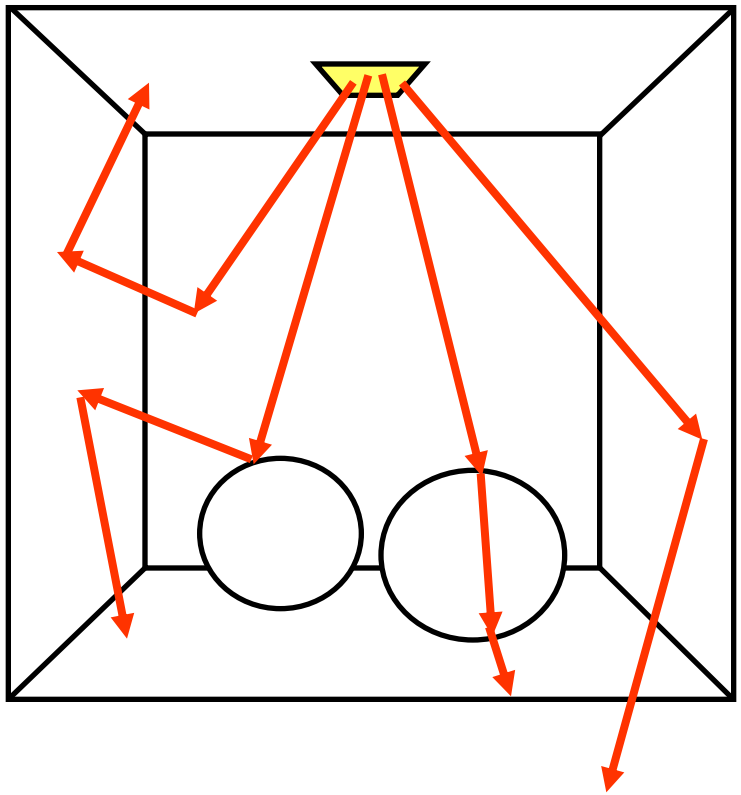
- Photon tracing. Emisión de fotones
- Se generan fotones desde la fuente desde las fuentes de luz. La distribución de los fotones dependerá del tipo de luz





3. Photon Mapping

- Photon tracing. Seguimiento de fotones
- Cuando un fotón choca contra una superficie puede ser absorbido, reflejado (reflexión especular o difusa) o transmitido



3. Photon Mapping

- Photon tracing. Seguimiento de fotones
- Para decidir el tipo de interacción de un fotón y que camino seguir se realiza un sorteo (ruleta rusa)
- Suponiendo una superficie con todas las propiedades y siendo k_a, k_d , k_s y k_t los coeficientes de absorción, reflexión difusa, especular y transmisión respectivamente

$$\mathrm{K_a} + K_d + K_s + K_t = 1 \qquad sea\ \ v\ \ un\ \ número\ \ aleatorio\ \ entre\ \ [0,1]$$

$$si$$

$$v \in [0, \mathrm{K_a}] \rightarrow\ absorción$$

$$v \in (\mathrm{K_a}, \mathrm{K_a} + K_d] \rightarrow\ reflexión\ \ difusa$$

$$v \in (\mathrm{K_a} + K_d, \mathrm{K_a} + K_d + K_s] \rightarrow\ reflexión\ \ especular$$

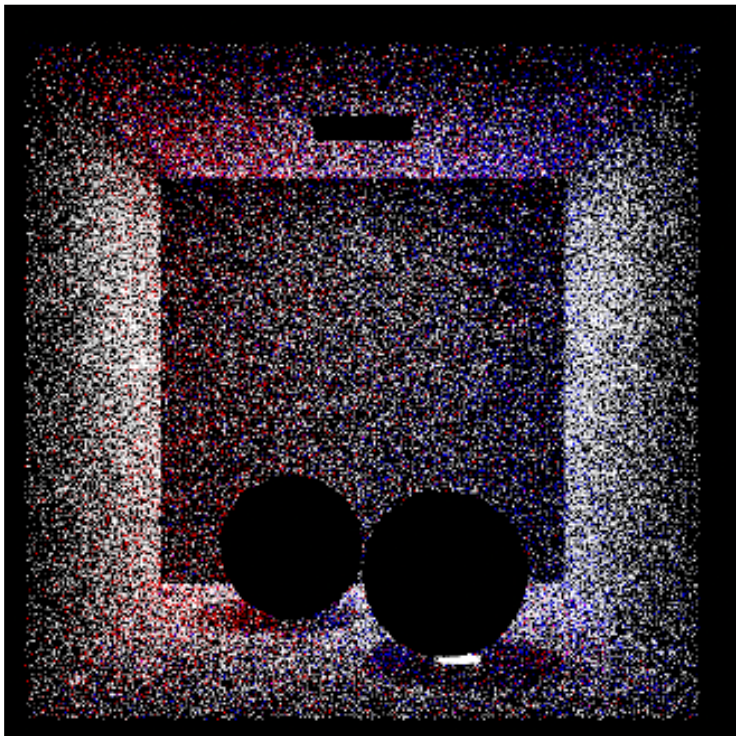
$$v \in (\mathrm{K_a} + K_d + K_s, 1] \rightarrow\ transmisión$$

3. Photon Mapping

- Photon tracing. Mapa de fotones
- Los fotones se almacenan únicamente cuando golpean una superficie difusa. De esta forma se genera un mapa de fotones de la escena
- Información que se almacena

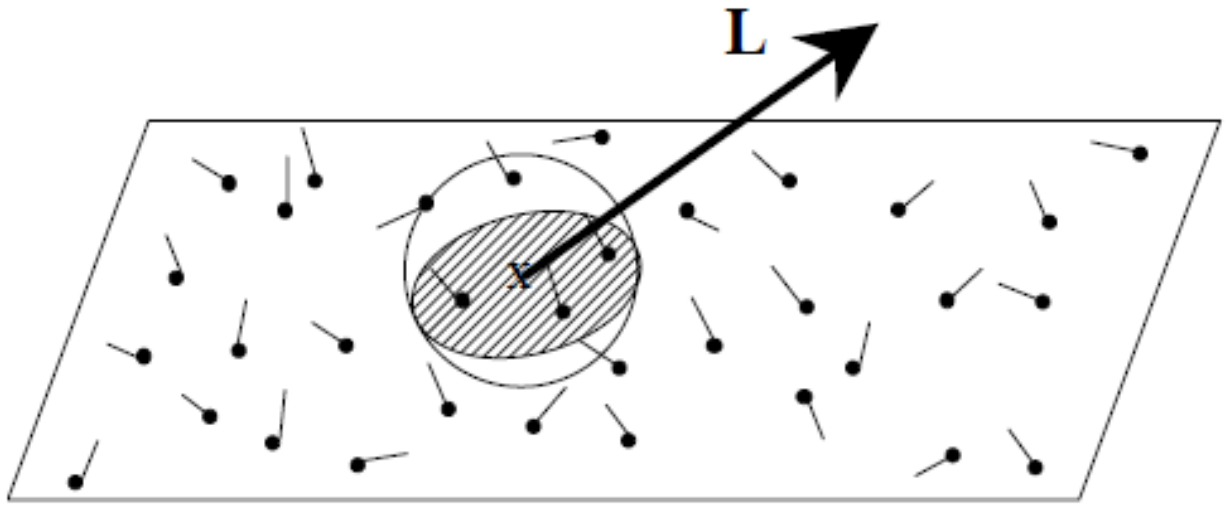
```
struct photon{
float x,y,z;          // posición
char rgb[3];          // color
char phi, theta; // dirección de incidencia
}
```

- Un fotón puede almacenar esta información varias veces. Se usa un kd-tree como estructura de datos



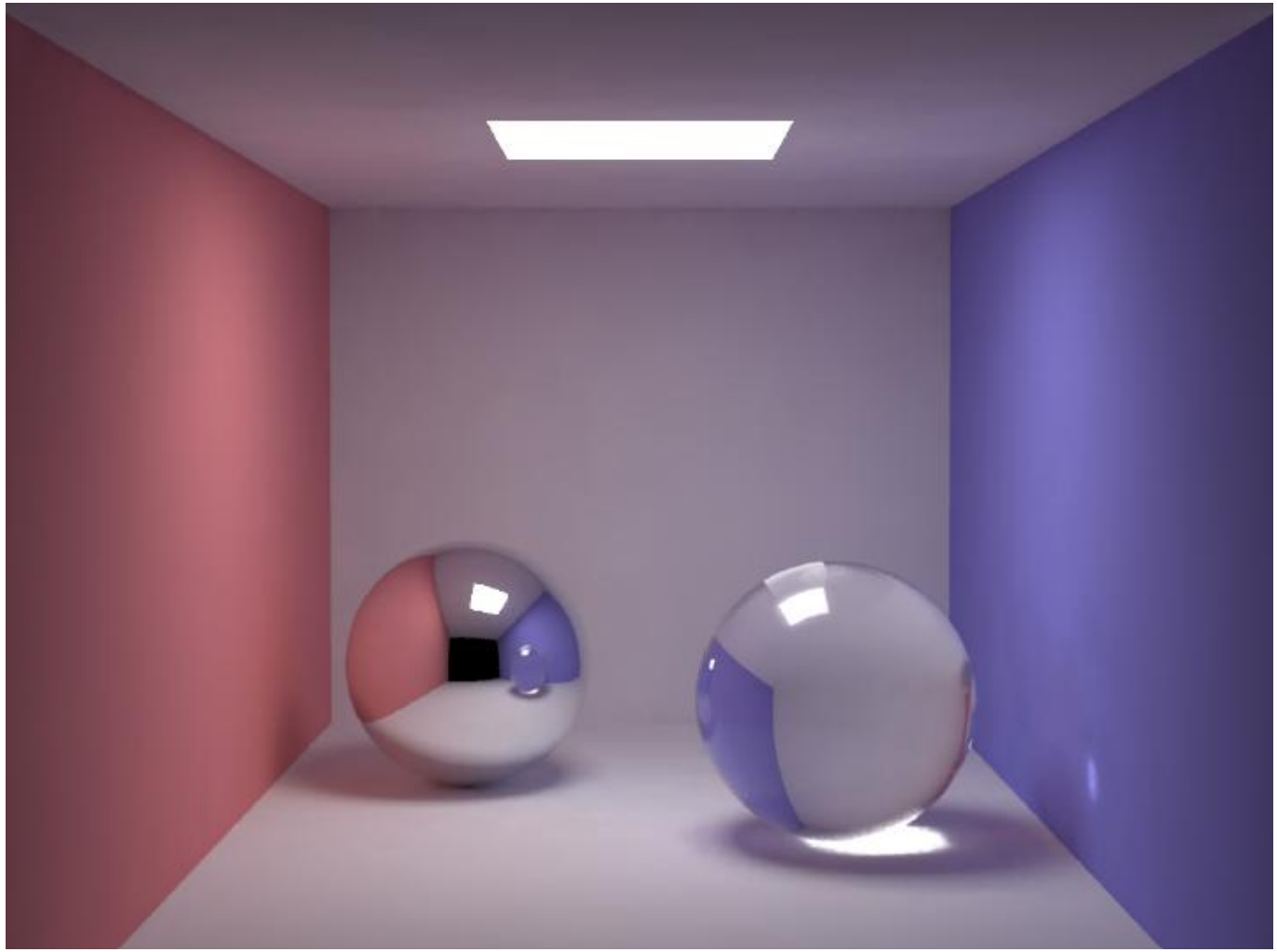
3. Photon Mapping

- Síntesis de la imagen
- Para encontrar el color en un punto se buscan los fotones más cercanos y se aplica la ecuación de rendering.



- Se aplica un filtro para disminuir el ruido de la imagen obtenida.

3. Photon Mapping

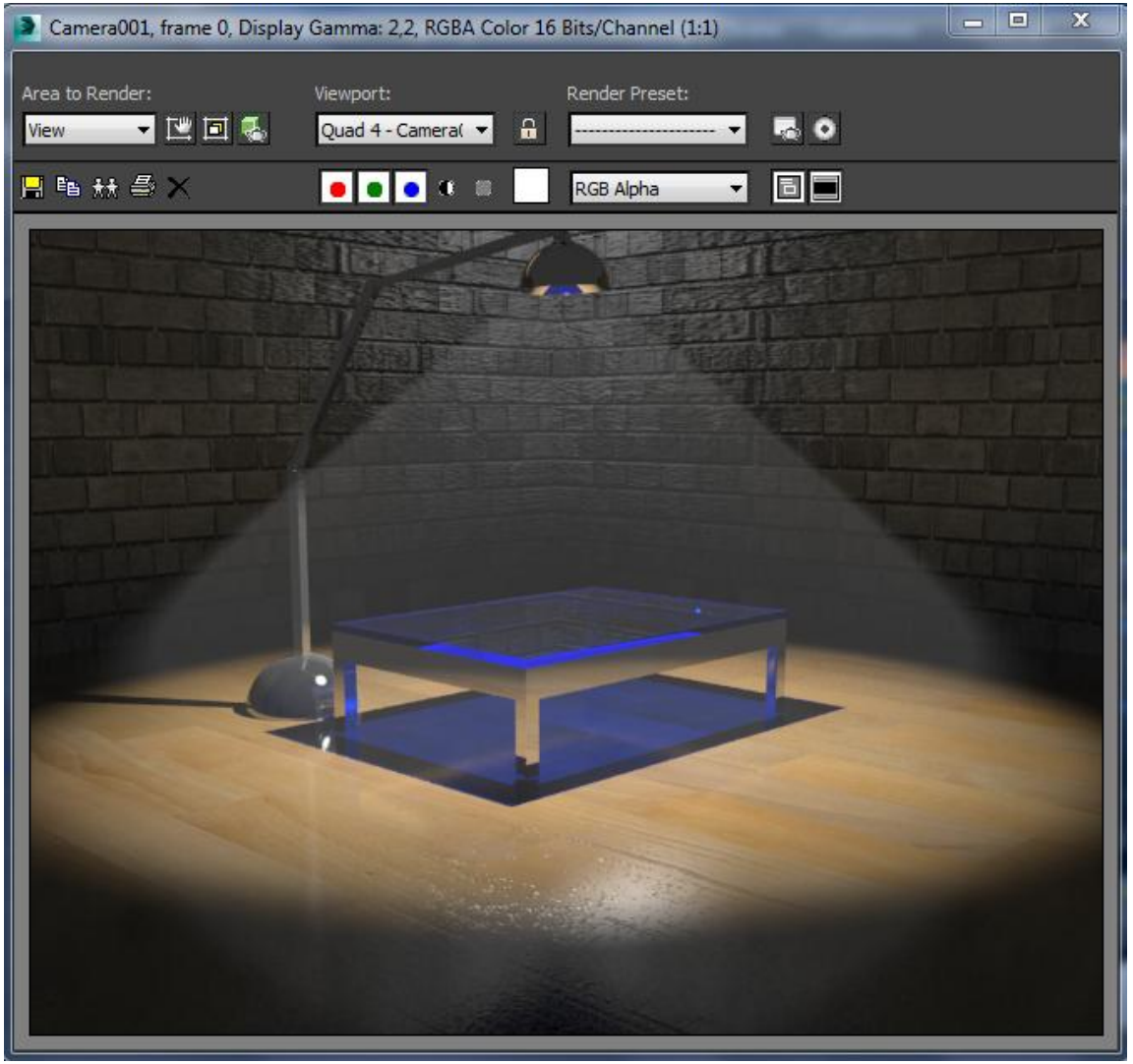


Jensen y Christensen Siggraph 2000

3. Photon Mapping

- Medios participativos. Los fotones pueden ser absorbidos o dispersados por el medio (volume photon map)
- Rayos crepúsculares o rayos de Dios

https://chrisfontas.wordpress.com/

3. Photon Mapping

- 3ds max